

Lisp in 99 lines of C and how to write one yourself

Dr. Robert A. van Engelen

July 9, 2022; updated August 24, 2026

“In 1960, John McCarthy published a remarkable paper in which he did for programming something like what Euclid did for geometry. He showed how, given a handful of simple operators and a notation for functions, you can build a whole programming language. He called this language Lisp, for “List Processing,” because one of his key ideas was to use a simple data structure called a list for both code and data.” – Paul Graham [1]

1 Introduction

McCarthy’s paper [2] not only showed how a programming language can be built entirely from lists as code and data, he also showed a function in Lisp that acts like an interpreter for Lisp itself. This function, called `eval`, takes as an argument a Lisp expression and returns its value. It was a remarkable discovery that Lisp can be written in Lisp itself. Lisp also introduced the concept of functions as first-class objects (*closures*) with *static scoping*¹, *runtime typing* and *garbage collection*. Features we now take for granted but were radical at that time. To put this into context, other programming languages at that time were Fortran (1957) and Algol (1958). Many new programming languages have appeared since. Most are still “Algol-like”, or as some say, “C-like.” The Lisp model of computation has regained momentum over the past decade. Contemporary programming languages now include Lisp-like “lambda functions.” Lambda functions are syntactic materializations of *lambda abstractions* from the *lambda calculus* [3] introduced by Alonzo Church in 1944. It was lambda calculus that inspired McCarthy to write Lisp.

In honor of the contributions made by Church and McCarthy, I wrote this article to show how anyone can write a tiny Lisp interpreter in a few lines of C or any “C-like” programming language. I attempted to preserve the original meaning and flavor of Lisp as much as possible. As a result, the C code in this article is strongly Lisp-like in compact form. Despite being small, these tiny Lisp interpreters in C include 21 built-in Lisp primitives, garbage collection and REPL, which makes them a bit more practical than a toy example. If desired, more Lisp features can be easily added with a few more lines of C as I will show in this article with examples that are ready for you to try.

I encourage anyone to explore other Lisp implementations and their code. Many are cool with lots of features. Some are actually incorrect. Sometimes little nuggets surface when digging deeper. This appears to be the case when writing this article, as it turns out that the list dot operator plays an important role in lambda variable lists and actual arguments lists. No special forms are needed to achieve the same power in a simple and pure form of Lisp, as was essentially discovered by McCarthy. *I hope you will enjoy reading this article as much as I did writing it!*

¹Originally *dynamic scoping*, which has some drawbacks. *Static scoping* is also known as *lexical scoping*.

Contents

1	Introduction	1
2	Understanding Lisp	5
3	Lisp Expressions as Tagged Structures	8
3.1	NaN Boxing	9
3.2	BCD Float Boxing	10
3.3	Types of Lisp Expressions	11
4	Constructing Lisp Expressions	13
5	Evaluating Lisp Expressions	16
6	Lisp Primitives	17
6.1	eval	18
6.2	quote	18
6.3	cons	18
6.4	car and cdr	19
6.5	Arithmetic	19
6.6	int	19
6.7	Comparison	19
6.8	Logic	19
6.9	cond	20
6.10	if	20
6.11	let*	20
6.12	lambda	21
6.13	define	21
7	Parsing Lisp Expressions	21
8	Printing Lisp Expressions	23
9	Garbage Collection	24
10	The Read-Eval-Print Loop	25
11	Additional Lisp Primitives	25
11.1	assoc and env	25
11.2	let and letrec	25
11.3	setq	26
11.4	set-car! and set-cdr!	26
11.5	macro	27
11.6	read and print	29
12	Adding Readline with History	30

13 Tracing Lisp	31
14 Adding Error Handling and Exceptions	32
14.1 C setjmp	32
14.2 Catching SIGINT to Stop Running Programs	34
14.3 Using C++ Exceptions	35
15 Downsizing Lisp to Single Floating Point Precision	35
16 Optimizing the Interpreter	37
16.1 Replacing Recursion with Loops	37
16.2 Tail-Call Optimization	38
16.3 Tail-Call Optimization Part Deux	40
16.4 Optimizing the Lisp Primitives	42
16.5 Tail-Calls à Trois	45
16.6 Higher-Order Functions, Tail-Recursive Functions and Loops	48
17 Mad Magical Macro Metaprogramming	50
17.1 Early Binding and Efficient Macro Expansion	52
17.2 Hygienic Macros	58
18 Simple String Literals	63
19 Conclusions	63
References	64
A Tiny Lisp Interpreter with NaN boxing: 99 Lines of C	65
B Tiny Lisp Interpreter with BCD boxing: 99 Lines of C	67
C Optimized Lisp Interpreter with NaN boxing	69
D Optimized Lisp Interpreter with BCD boxing	72
E Example Lisp Functions and Macros	75
E.1 Standard Lisp Functions	75
E.2 Math Functions	76
E.3 List Functions	76
E.4 Other Higher-Order Functions	80
E.5 Revealing Closures	80
E.6 Macros	81
F List of Additions, Edits and Corrections	83

Copyright (c) Robert A. van Engelen, 2022. Permission to share all or any parts of this work is hereby granted, provided that this article is cited as the source. “Tinylisp” project code is licensed BSD-3. Authenticity: no AI was used in the creation of this publication and source code.

Biography Dr. Robert A. van Engelen received a M.S. in Computer Science from Utrecht University, the Netherlands, in 1994 and the Ph.D. in Computer Science from the Leiden Institute of Advanced Computer Science (LIACS) at Leiden University, the Netherlands, in 1998. As tenured professor in Computer Science and Scientific Computing, he served 20 years at the department of Computer Science at the Florida State University, where he also served as department chair. During these years he taught university-accredited undergraduate and graduate-level courses on Programming Languages, Foundations of Programming Languages, Compiler Construction, High-Performance Computing, and other courses. His research interests include High-Performance Computing, Programming Languages and Compilers, Problem-Solving Environments for Scientific Computing, Cloud Computing, Services Computing, Machine Learning, and Bayesian Networks. Van Engelen's research has been recognized with awards and research funding from the US National Science Foundation and the US Department of Energy. Van Engelen is a member of the ACM and IEEE professional societies. He currently is the CEO/CTO of Genivia.com, a US technology company he founded in 2003.

2 Understanding Lisp

Lisp programs are composed of anonymous functions written in the form of a ()-delimited list

```
(lambda variables expression)
```

where *variables* is a list of names denoting the function parameters and *expression* is the body of the function. Just like any other ordinary math function, lambdas don't do anything until we apply them to arguments. Application of a lambda to arguments is written as a list

```
(function arguments)
```

The application is performed in two steps. First, we bind the *variables* to the values of the corresponding *arguments*. Then the *expression* is evaluated. The *expression* may reference the function's *variables* by their name. The value of *expression* is “returned” as the result of the application.

Note that “return” is an imperative concept. Lisp has no imperative keywords. The entire Lisp language is built from functions and function applications, all using lists as syntax. Besides lists, Lisp also has symbols (names) for variables, primitives, functions (closures) and numbers. Lisp dialects may also include strings.

Function application is perhaps best illustrated with an example. Consider the function

```
(lambda (x y) (/ (- y x) x))
```

This function returns the value of $\frac{y-x}{x}$ for numeric arguments x and y . The first thing we note is that all arithmetic operations are written in functional form in Lisp. There is no need for any specific rules for operator precedence and associativity in Lisp. To apply our lambda to arguments, say 3 and 9, we write the list

```
((lambda (x y) (/ (- y x) x)) 3 9)
```

Spacing in Lisp is immaterial, so let's add some more spacing

```
( ( lambda (x y)
  (/ (- y x) x)
)
  3 9
)
```

The Lisp interpreter binds `x` to 3 and `y` to 9, then evaluates the function body `(/ (- y x) x)` to compute 2 as the result of the application. Nice, isn't it?

But our function is not stored anywhere. What if we want to reuse it? After all, programs are composed of functions and those functions should be stored as part of a program to use them². We can save our function by giving it a name using a **define**

```
(define subdiv (lambda (x y) (/ (- y x) x)))
```

and then apply `subdiv` to 3 and 9 with

```
(subdiv 3 9)
```

²The beauty of lambda calculus is that this is not an absolute requirement: lambda calculus is Turing-complete without named functions.

which displays 2. A defined name is not required to be alphabetic. Names are syntactically symbolic forms in Lisp. We could have named our lambda `-/` for example. Any sequence of characters can be used as a name, as long as it is distinguishable from a number and doesn't use parenthesis, quotes and whitespace characters.

The power and simplicity of Lisp's lambdas is better justified when we take a closer look at *closures*. A closure combines a function (a lambda) with an *environment*. An environment defines a set of name-value bindings. An environment is created (or extended) when the variables of a lambda are bound to the argument values in a lambda application. An environment provides a concrete mechanism to create a local scope of variables for the function body. Because lambdas are first-class objects and can therefore be returned as values by lambdas, environments play a crucial role to scope nested lambdas properly through *static scoping*. Consider for example the `make-adder` lambda that takes an `x` to return a new lambda that takes a `y` and adds them together:

```
(define make-adder (lambda (x) (lambda (y) (+ x y))))
```

Applying `make-adder` to 5 returns a closure in which the environment includes a binding of `x` to 5. When this closure is applied to 2 it returns 7 as expected:

```
> (define make-adder (lambda (x) (lambda (y) (+ x y))))
> ((make-adder 5) 2)
7
> (define add5 (make-adder 5))
> (add5 2)
7
```

Note that `make-adder` returns a closure that combines `(lambda (y) (+ x y))` with an environment in which `x` is bound to 5. The `x` in the lambda body is not modified. It is Lisp code after all. When the closure is applied, the environment is extended to include a binding of `y` to 2. With this environment the function body `(+ x y)` evaluates to 7.

A handful of programming languages both correctly and safely implement the semantics of closures with static scoping. The implementation requires *unlimited extent* of non-local variables in scope to store their name-value bindings. Otherwise, non-local variables are “gone” as their values are removed from memory. This requires environments and garbage collection to remove them safely after the work is done. It doesn't suffice that functions can be syntactically nested within other functions.

Lisp also has a collection of built-in *primitives*. These are functions like `+` and *special forms* like `define`. A special form is a function that selectively evaluates its arguments rather than all of its arguments as in lambda applications. For example, `define` does not evaluate its *name* argument. Otherwise the value of the *name* would end up being used by `define` or an error is produced when *name* is not yet defined, which is more likely.

An overview of Lisp is not complete without a presentation of the basic primitives introduced in McCarthy's paper. We list them here and also include two more primitives `if` and a `let` since these are often used in Lisp³. Some of the primitives listed below are special forms, namely `quote`, `cond`, `if` and `let`:

³The `if` and `let` can be defined as macros, but we keep our Lisp interpreter small without macro processing. To add macro processing, see Section 11.5.

- `(quote x)` returns x unevaluated, “as is”. Abbreviated `'x`.

```
> (quote a)
a
> 'a
a
> '(a b c)
(a b c)
```

- `(cons x y)` returns the pair $(x . y)$ where the dot is displayed if y is not the empty list `()`.

```
> (cons 'a 'b)
(a . b)
> (cons 'a ())
(a)
> (cons 'a (cons 'b (cons 'c ())))
(a b c)
```

- `(car x)` (“*Contents of the Address part of Register*”) returns the first element of the pair or list x , i.e. the *head* of the list.

```
> (car (cons 'a 'b))
a
> (car (cons 'a (cons 'b (cons 'c ())))))
a
```

- `(cdr x)` (“*Contents of the Decrement part of Register*”, pronounced “coullder”) returns the second element of the pair x . When x is a non-empty list, the rest of the list is returned after the first element, i.e. the *tail* of the list.

```
> (cdr (cons 'a 'b))
b
> (cdr (cons 'a (cons 'b (cons 'c ())))))
(b c)
```

- `(eq? x y)` returns the atom `#t` (representing true) if the values of x and y are identical. Otherwise returns `()` representing false.

```
> (eq? 2 2)
#t
> (eq? 2 3)
()
> (eq? 'a 'a)
#t
```

- `(pair? x)` returns the atom `#t` (representing true) if x is a pair, that is, a non-empty list cons cell to which `car` and `cdr` can be applied. Otherwise returns `()` representing false.

```

> (pair? '(1 2))
#t
> (pair? '(1))
#t
> (pair? ())
()

```

- `(cond (x1 y1) (x2 y2) ... (xn yn))` evaluates x_i from left to right until x_i is not the empty list (i.e. is true), then returns the corresponding value of y_i .

```

> (cond ((eq? 'a 'b) 1) ((eq? 'b 'b) 2))
2
> (cond (() 1) (#t 2))
2

```

- `(if x yt yf)` if x is not the empty list (i.e. is true), then the value of y_t is returned else the value of y_f is returned. `(if x yt yf)` is a shorthand for `(cond (x yt) (#t yf))`.

```

> (if 'a 1 2)
1
> (if () 1 2)
2
> (if (eq? 'a 'a) 'ok 'fail)
ok

```

- `(let ((v1 x1) (v2 x2) ... (vn xn)) y)` evaluates x_i from left to right and binds each variable v_i to the value of x_i to extend the environment to the body y , then returns the value of y . The same is accomplished with `((lambda (v1 v2 ... vn) y) x1 x2 ... xn)`.

```

> (let ((x 3) (y 9)) (/ (- y x) x))
2

```

The `let` special forms in this article do not require the bindings list. For example, we will write `(let (x 3) (y 9) (/ (- y x) x))` which works the same way but is more compact.

Lisp implementations include many more primitives, notably for arithmetic, logic and runtime type checking. Most Lisp implementations define additional Lisp primitives in Lisp itself. We will exactly do that for our tiny Lisp too, see the Appendices.

3 Lisp Expressions as Tagged Structures

Lisp expressions are composed of numbers, atoms (names and symbols), strings (when implemented), primitives, cons pairs and closures. A Lisp expression type can be conveniently defined in C as a tagged union:

```

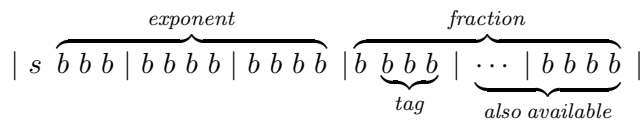
struct Expr {
  enum { NMBR, ATOM, STRG, PRIM, CONS, CLOS, NIL } tag;
  union {
    double number;           /* NMBR: double precision float number */
    const char *atom;        /* ATOM: pointer to atom name on the heap */
    const char *string;      /* STRG: pointer to string on the heap */
    struct Expr (*fn)(struct Expr, struct Expr); /* PRIM: built-in primitive */
    struct Expr *cons;       /* CONS: pointer to (car,cdr) pair on the heap */
    struct Expr *closure;    /* CLOS: pointer to closure pairs on the heap */
  } value;
};

```

However, rather than storing this information elaborately in a structure, we can exploit *NaN boxing* to store this information in an IEEE-754 single or double precision float, because all structure members are pointers that are essentially unsigned integer offsets from a base address.

3.1 NaN Boxing

The idea behind NaN boxing is that IEEE 754 floating point NaN (“Not-a-Number”) values are not unique. A double precision NaN allows up to 52 bits to be arbitrarily used to stuff any information we want into a double precision NaN:



where

s is the sign bit of the float, $s = 1$ for negative numbers

exponent consists of 11 bits to represent binary exponents -1022 to 2023, when the bits are all 1 the value is NaN or INF

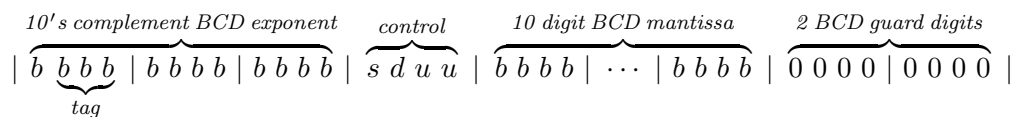
fraction consists of 52 bits with an invisible 1 bit as the leading digit of the mantissa

The *tag* and other data can be stored in the freely available 52 bit fraction part of a NaN. However, we want to use *quiet NaNs* which means that the first bit of the fraction (the bit before the *tag* bits) must be 1, leaving 51 bits and the sign bit for both the *tag* and other data to our disposal. This is plenty of space in a NaN-boxed double precision float to store a tag to identify atoms, strings, primitives, cons pairs, and closures together with their pointers and/or integer indices. In all, 48 bits are available to store an integer, or 49 bits when including the sign bit.

In this article I will also describe a Lisp implementation for the Sharp PC-G850 vintage pocket computer, which poses a bit of a challenge since it does not use IEEE 754 floating point representations. Instead, floating point values are represented internally in BCD (Binary Coded Decimal). This raises the question: *can we use similar tricks as NaN boxing with BCD floats?* Let’s find out.

3.2 BCD Float Boxing

To understand how we can exploit BCD floats to store information other than floating point numbers, we will take a closer look at the PC-G850's internal decimal floating point representation. A decimal floating point value is stored in 8 bytes. The first 2 bytes store the exponent in BCD and the sign of the number. The next 5 bytes store the 10 digit BCD mantissa followed by a zero byte:



where

s is the sign bit of the float, $s = 1$ for negative numbers

d is the degree bit, $d = 1$ to display degrees in $D^{\circ}M'S.S''$ format in BASIC

u is an unused bit, likely a mantissa carry bit used by the system

10's complement BCD exponent consists of 12 bits for 3 BCD digits to represent exponents -99 (901 BCD) to 99 (099 BCD)

10 digit BCD mantissa consists of 10 BCD digits with the normalized mantissa, the leading BCD digit is nonzero unless all mantissa digits are zero

2 BCD guard digits are always zero after internal rounding to 10 significant digits

To explore opportunities to exploit *BCD float boxing*⁴ to store information other than decimal floating point numbers, we can write some C code for testing. Unfortunately, we cannot use the mantissa or its trailing *guard digits* to store extra information. The mantissa is always normalized to BCD and the guard digits are always reset to zero when passing floats through functions, even when no arithmetic operations are applied to the float. Our target bits to box a *tag* with data in a float are the three bits in the upper half of the leading byte of the float. These three bits of the float remain unmodified when passing the floating point value through functions as arguments and as return values. A quick test confirms our hypothesis, with some caveats:

```
double func(double x) { return x; }
int main() {
    double x,y,z; char *p = (char*)&x,*q = (char*)&y; int i;
    scanf("%lg",&z);                               /* input a value z to check */
    for (i = 0x20; i <= 0x70; i += 0x10) {
        x = z; *p |= i;                             /* set x to z and set its tag bits */
        x = func(x);                                /* pass x through func() */
        y = z; *q |= i;                             /* set y to z and set its tag bits */
        if (x != y)                                 /* x and y should be equal */
            printf("fail %x\n",i);
    }
}
```

⁴I think the term "BCD float boxing" nails it, but Google seems to think it's sports gear.

The first caveat is that tag 000 ($i == 0x00$) cannot be used. This tagged value is indistinguishable from a normal float value. Second, tag 001 ($i == 0x10$) cannot be used because all tagged floats appear to fail with an arithmetic error when passed to a function, perhaps because the tagged value corresponds to a non-normalized carry digit in the exponent. Third, all tagged floats with values $|x| < 10$ are normalized to zero and thus fail this test. This type of failure happens when the two-digit BCD exponent is zero and the third highest order BCD exponent digit nonzero, thus representing a non-normalized two-digit zero BCD exponent.

After confirming our hypothesis by observation, we conclude that we have six possible tag bit patterns 010 to 111 to our disposal, as long as we box integers as tagged floats $|x| \geq 10$. This is not a problem, because we can simply multiply a value by 10 before boxing and divide by 10 after unboxing. When boxing unsigned integers, such as pointers and array indices, it suffices to add 10 before boxing and subtract 10 after unboxing. So we will use this simpler boxing method.

3.3 Types of Lisp Expressions

We define two types⁵ in C that we exclusively use in our Lisp interpreter: a type I for unsigned integers and a type L for Lisp expressions defined as floats for NaN or BCD float boxing:

```
#define I unsigned
#define L double
```

We define five tags for ATOMs, PRIMitives, CONStructed pairs, CLOSures and NIL (the empty list):

```
/** NaN Boxing */
I ATOM=0x7ff8, PRIM=0x7ff9, CONS=0x7ffa, CLOS=0x7ffb, NIL=0x7ffc;
/** BCD boxing */
I ATOM=32, PRIM=48, CONS=64, CLOS=80, NIL=96;
```

To access the tag bits of a tagged NaN float we shift the 48 lower bits out to return the 16-bit tag:

```
/** NaN Boxing with old-style pointer casting */
#define T(x) *((uint64_t*)&x)>>48
/** NaN Boxing in modern C using a safe union */
I T(L x) { union { L x; uint64_t i; } u = {x}; return u.i>>48; }
/** BCD boxing */
#define T *((char*)&
```

Instead of `uint64_t` defined in `stdint.h` we can use `unsigned long long` that is typically 64 bits⁶. We also need the following two functions to manipulate tagged floats and their ordinal content:

```
/** NaN Boxing with old-style pointer casting */
L box(I t, I i) { L x; *((uint64_t*)&x) = (uint64_t)t<<48|i; return x; }
I ord(L x) { return *((uint64_t*)&x; }
/** NaN Boxing in modern C using a safe union */
```

⁵PC-G850 C does not support `typedef`. We use `#define` instead.

⁶At least 64 bits, but we want 64 exactly. C is one of the oldest programming languages and had to accommodate systems with strange bit widths like 18 and 36 and so on, hence “at least”.

```

L box(I t,I i) { union { uint64_t i; L x; } u = {(uint64_t)t<<48|i}; return u.x; }
I ord(L x) { union { L x; uint64_t i; } u = {x}; return u.i; }
  /** BCD boxing */
L box(I t,I i) { L x = i+10; T(x) = t; return x; }
I ord(L x) { T x &= 15; return (I)x-10; }

```

The `box` function returns a float tagged with the specified tag `t` as `ATOM`, `PRIM`, `CONS`, `CLOS` or `NIL` and by boxing unsigned integer `i` as ordinal content. For BCD boxing we must add 10 to `i` to avoid the aforementioned caveat when boxing values in BCD floats. The `ord` function unboxes the unsigned integer (ordinal) of a tagged float. For BCD boxing we first untag the float with `T x &= 15` then subtract 10 to return the boxed ordinal content of the tagged float.

We should be able to perform arithmetic on floats in our Lisp interpreter. To do so, we could simply assume that the arguments and operands to arithmetic operations are always untagged floats. However, to make sure we aren't applying arithmetic operations on tagged floats by accident, we should define a new function `num` to check or clear tag bits first, before applying arithmetic operations:

```

  /** NaN Boxing */
L num(L n) { return n; }
  /** BCD boxing */
L num(L n) { T n &= 159; return n; }

```

With NaN boxing, the decimal number is returned “as is”, but we could check if `n` is a NaN⁷ and take some action. For now, we just pass NaNs to perform arithmetic on, which results in a NaN. For BCD boxing we clear two bits of the tag with `T n &= 159 (0x9f)` since negative BCD exponents are represented in BCD *9dd*. The high-order digit 9 (binary 1001) should be preserved.

Checking if two values are equal is performed with the `equ` function. Because equality comparisons `==` with NaN values always produce false, we compare the raw 64 bits of the two floats for equality:

```

  /** NaN Boxing with old-style pointer casting */
I equ(L x,L y) { return *(uint64_t*)&x == *(uint64_t*)&y; }
  /** NaN Boxing in modern C using a safe union */
I equ(L x,L y) { union { L x; uint64_t i; } u = {x},v = {y}; return u.i == v.i; }
  /** with BCD boxing */
I equ(L x,L y) { return x == y; }

```

Note that BCD boxing does not really require defining an `equ` function. It just wraps `==`. But we include it here since we may have to redefine it depending on the BCD arithmetic performed by a specific machine.

Checking if a Lisp expression is `nil` (the empty list) only requires checking its tag for `NIL`:

```

I not(L x) { return T(x) == NIL; }

```

The `not` function comes in handy later when we implement conditionals, since `nil` is considered *false* in Lisp. Anything else is implicitly *true* in Lisp.

The C functions we defined here are the only ones specific to NaN or BCD float boxing. The rest of our Lisp interpreter is independent of the tagging method used.

⁷Checking if a value `n` is a NaN is easy, just do an equality check on itself: `if (n != n) NaN-action-here.`

4 Constructing Lisp Expressions

Lisp expressions are composed of atoms (also called symbols in Lisp), primitives, cons pairs, closures and `nil`. The `nil` constant represents the empty list `()` in Lisp. The `nil` constant is also considered *false* in Lisp conditionals. Furthermore, we have two pre-defined atoms, namely `#t` and `ERR`. The `#t` atom will be used as an explicit *true* in Lisp, although any value other than `nil` is implicitly *true* in Lisp conditionals. The `ERR` atom represents an error and is returned to the user when an expression evaluates to an error. These constants are globally declared since we will often use them in the internals of our Lisp interpreter. They are initialized⁸ in `main` as follows:

```
L nil,tru,err;
...
int main() {
    ...
    nil = box(NIL,0); err = atom("ERR"); tru = atom("#t");
    ...
}
```

where we used the `box` function defined in Section 3.2. The `atom` function returns an `ATOM`-tagged float that is globally unique. The `atom` function checks if the atom name already exists on the heap and returns the heap index corresponding to the atom name boxed in the `ATOM`-tagged float. If the atom name is new, then additional heap space is allocated to copy the atom name into the heap as a string. The heap index of the new atom name is boxed in the `ATOM`-tagged float and returned by the `atom` function:

```
L atom(const char *s) {
    I i = 0; while (i < hp && strcmp(A+i,s)) i += strlen(A+i)+1;
    if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<3) abort();
    return box(ATOM,i);
}
```

where `hp` is the *heap pointer* pointing to free bytes available on the heap. When `i == hp` we execute `hp += strlen(strcpy(A+i,s))+1` to copy the string `s` to the free heap address at `A+hp` and return the length of the string plus one to increase `hp`. `A` is the starting byte address of the heap and `sp` is the *stack pointer* pointing to the top of the stack of Lisp values (tagged and untagged floats `L`):

```
#define A (char*)cell
#define N 1024
L cell[N];
I hp = 0,sp = N;
```

The `cell[N]` array of 1024 (tagged) floats contains both the heap and the stack. The value of `N` can be increased to pre-allocate more memory.

The `atom` function searches the heap at addresses `A+i` until a matching atom name is found to return `box(ATOM,i)`. If the atom name is new, then space for the atom's string name is allocated

⁸We initialize globals in `main` since PC-G850 C does not support initialization of globals with non-constants, i.e.. function calls cannot be used as initializers of globals.

and copied into this space with a terminating zero byte. In this way, atoms constructed with `box(ATOM,i)` are globally unique. The method by which we construct them by looking them up in a pool of names is often referred to as *interning*.

The heap grows upward towards the stack. The stack grows downward, as stacks usually do. The remaining free space is available between the heap and stack. For example, the table below depicts the memory configuration after pushing a pair of cells `box(ATOM,4)` and `nil` on the stack and storing two atoms `ERR` and `#t` in the heap:

cell[1023]:	box(ATOM,4)	stack
cell[sp=1022]:	nil	↓
	free	
A+hp=A+7:	space	
A+4:	"#t\0"	↑
A+0:	"ERR\0"	heap

What's actually on the stack here is the Lisp list (`#t`) containing one element `#t` in the list. This list is represented by `box(CONS,1022)` where 1022 is the stack index of the `cdr` cell of this list pair. The cell above it on the stack contains the `car` of this list pair. Lisp uses linked lists with the `car` of a list node (a cons pair) containing the list element and `cdr` pointing to the next cons pair in the list or it is `nil` at the end of the list.

With this memory configuration in mind, constructing cons pairs is easy. We just allocate two cells on the stack, copy the values therein and return `box(CONS,sp)` since `sp` points to the `cdr` cell:

```
L cons(L x,L y) {
  cell[--sp] = x;
  cell[--sp] = y;
  if (hp > sp<<3) abort();
  return box(CONS,sp);
}
```

If the `hp` and `sp` pointers meet we ran out of memory and we should `abort` or take some other action. Note that `hp` points to bytes whereas `sp` points to 8-byte floats. Therefore, the out-of-memory condition is checked in the `atom` and `cons` functions by scaling `sp` by a factor 8 in the conditional `if (hp > sp<<3) abort()`.

Deconstructing a cons pair is trivial. We just need to get the `cell` of the `car` or `cdr` indexed by `i` in `box(CONS,i)` by retrieving it with the `ord` function:

```
L car(L p) { return (T(p) & ~(CONS^CLOS)) == CONS ? cell[ord(p)+1] : err; }
L cdr(L p) { return (T(p) & ~(CONS^CLOS)) == CONS ? cell[ord(p)] : err; }
```

where `ord(p)` is the cell index of the `cdr` of the cons pair `p` with the `car` cell located just above it. However, we do not trust the argument `p` to be a cons pair. The condition `T(p) & ~(CONS^CLOS)) == CONS` guards valid `car` and `cdr` function calls on cons pairs. The condition is true if `p` is a cons pair or a closure pair. Closure pairs are just cons pairs tagged as closures. The `~(CONS^CLOS)` mask is an efficient way to check for both `CONS` and `CLOS` tags in one comparison, because the tag values of `CONS` and `CLOS` were carefully chosen to differ by only one bit.

For example, suppose $p = \text{box}(\text{CONS}, \text{sp})$ representing the list $(\#t)$ with the cells on the stack depicted in the memory configuration shown previously. Then $\text{car}(p)$ returns $\text{box}(\text{ATOM}, 4)$ representing $\#t$ and $\text{cdr}(p)$ returns nil , the empty list.

Closures and environment lists are constructed with the `cons` function applied twice, first to construct the name-value Lisp pair⁹ $(v . x)$, then to place the pair in front of the Lisp environment list. We define a `pair` function for this purpose:

```
L pair(L v, L x, L e) { return cons(cons(v,x),e); }
```

A closure is a CLOS-tagged `pair(v,x,e)` representing an instantiation of a Lisp `(lambda v x)` with either a single atom v as a variable referencing a list of arguments passed to the function, or v is a list of atoms as variables, each referencing the corresponding argument passed to the function. Closures include their static scope as an environment e to reference the bindings of their parent functions, if functions are nested, and to reference the global static scope:

```
L closure(L v, L x, L e) { return box(CLOS, ord(pair(v,x, equ(e, env) ? nil : e))); }
```

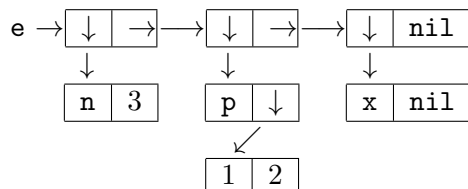
The conditional `equ(e, env) ? nil : e` forces the scope of a closure to be `nil` if e is the global environment `env`. Later, when we apply the closure, we check if its environment is `nil` and use the current global environment `env` to apply the closure. This permits a global function to make recursive calls and calls to forward-defined global functions. Some other Lisp may use full *dynamic scoping* for global functions in which also local variables are visible when the global function is called, which we can implement by using the current environment e instead of `env` in function `reduce` introduced in the next section. But dynamic scoping has caveats. It may not be safe to rename the names of arguments and local variables that may or may not be captured by dynamically-scoped functions that are called that reference outer-scope variables.

An *environment* in Lisp is implemented as a list of name-value associations, where names are Lisp atoms. Environments are searched with the `assoc` function given an atom v and an environment e :

```
L assoc(L v, L e) {
  while (T(e) == CONS && !equ(v, car(car(e)))) e = cdr(e);
  return T(e) == CONS ? cdr(car(e)) : err;
}
```

The `assoc` function returns the Lisp expression associated with the specified atom in the specified environment e or returns `err` if not found.

Consider for example the Lisp environment $e = ((n . 3) (p . (1 . 2)) (x . \text{nil}))$ where the Lisp expression $(1 . 2)$ constructs a pair of 1 and 2 instead of a list. This example environment e is represented in memory as follows with boxes for cells and arrows for `CONS` cells pointing to cells:



Note that a `CONS` index points to two cells on the stack, the `car` and `cdr` cells.

⁹The Lisp pair denoted with a dot $(v . x)$ is constructed by the Lisp parser with $p = \text{cons}(v, x)$, whereas a Lisp list always ends in `nil`, such as $(1\ 2)$ constructed by the Lisp parser in C with `cons(1, cons(2, nil))`.

5 Evaluating Lisp Expressions

A Lisp expression is either a number, an atom, a primitive, a cons pair, a closure, or nil. Numbers, primitives, closures and nil are constant and returned by `eval` as is. Atoms are evaluated by returning their associated value from the environment using the `assoc` function, see Section 4. The environment includes function parameters and global definitions. Lists are evaluated by applying the first element in the list as a function to the rest of the list as arguments passed to that function:

```
L eval(L x,L e) {
  return T(x) == ATOM ? assoc(x,e) :
         T(x) == CONS ? apply(eval(car(x),e),cdr(x),e) :
         x;
}
```

Note that Lisp expression `x` evaluates to the value `assoc(x,e)` of `x` when `x` is an atom, or evaluates to the result of a function application `apply(eval(car(x),e),cdr(x),e)` if `x` is a list, or evaluates to `x` itself otherwise. A function application requires evaluating the function `eval(car(x),e)` first before applying it, because `car(x)` may be an expression that returns a function such as an atom associated with a Lisp primitive or the closure constructed for a `lambda`. The `apply` function applies the primitive or the closure `f` to the list of arguments `t` in environment `e`:

```
L apply(L f,L t,L e) {
  return T(f) == PRIM ? prim[ord(f)].f(t,e) :
         T(f) == CLOS ? reduce(f,t,e) :
         err;
}
```

The `prim[]` array contains all pre-defined primitives. A primitive is stored in a structure with its name as a string and a function pointer to its code. To call the primitive we invoke the function pointer with `prim[ord(f)].f(t,e)`. See also Section 6. If `f` is a closure then `reduce`¹⁰ is called to apply closure `f` to the list of arguments `t`:

```
L reduce(L f,L t,L e) {
  return eval(cdr(car(f)),bind(car(car(f)),evlis(t,e),not(cdr(f)) ? env : cdr(f)));
}
```

where `f` is the closure and `t` is the list of arguments. The outer `eval` evaluates the body of closure `f` retrieved with `cdr(car(f))`. The evaluation is performed with an extended environment produced with the `bind` function. This extended environment includes the bindings of the lambda's variables to the evaluated arguments. Remember that a closure was constructed for a `lambda` to include its static scope or nil as its environment? We retrieve its environment with `cdr(f)` and check it for nil with `not(cdr(f)) ? env : cdr(f)`, which gives the current global environment `env` or the lambda's local static environment `cdr(f)`. This environment is used with `bind` to bind the lambda variables `car(car(f))` with the evaluated arguments `evlis(t,e)`.

Arguments passed to a function or primitive are evaluated with the `evlis` function:

¹⁰Viz. lambda calculus *beta reduction* involves a *contraction* step $(\lambda v.x) y \Rightarrow x[v := y]$ where y may or may not be evaluated first before the contraction. This models *strict* and *lazy evaluation*, respectively.

```

L evlis(L t,L e) {
    return T(t) == CONS ? cons(eval(car(t),e),evlis(cdr(t),e)) :
        T(t) == ATOM ? assoc(t,e) :
        nil;
}

```

where `evlis` recursively traverses the list of expressions `t` to create a new list with their values. Recursion bottoms out at a non-CONS `t`. It is important to correctly handle the dot operator in a list of actual arguments in this way, such as `(f x . args)`, by calling `eval(t,e)` to evaluate `args`.

The variable-argument bindings for `apply` are constructed as a list of pairs with the `bind` function originally called `pairlis`:

```

L bind(L v,L t,L e) {
    return T(v) == NIL ? e :
        T(v) == CONS ? bind(cdr(v),cdr(t),pair(car(v),car(t),e)) :
        pair(v,t,e);
}

```

where `v` is a list of variables or a variable (an atom), and `t` is the list of evaluated arguments. When recursion bottoms out, we either have a NIL or a name. The name is bound to the rest of the list `t` with `pair(v,t,e)`. The latter happens when a single variable is used like `(lambda args args)` and when a dot is used in the list of variables like `(lambda (x . args) args)`.

6 Lisp Primitives

The Lisp primitives are defined in an array `prim[]` of structures containing the name of the primitive as string `s` and the function pointer `f` pointing to the implementation in C. The function implementing the primitive takes the list of Lisp arguments as the first parameter and the Lisp environment as its second parameter:

```

struct { const char *s; L (*f)(L,L); } prim[] = {
    {"eval", f_eval},
    {"quote", f_quote},
    {"cons", f_cons},
    {"car", f_car},
    {"cdr", f_cdr},
    {"+", f_add},
    {"-", f_sub},
    {"*", f_mul},
    {"/", f_div},
    {"int", f_int},
    {"<", f_lt},
    {"eq?", f_eq},
    {"pair?", f_pair},
    {"or", f_or},
    {"and", f_and},

```

```

{"not",    f_not},
{"cond",   f_cond},
{"if",     f_if},
{"let*",   f_let},
{"lambda", f_lambda},
{"define", f_define},
{0}};

```

The main program initializes the global environment `env` with `#t` to return itself, followed by the Lisp primitives:

```

int main() {
    ...
    env = pair(tru,tru,nil);
    for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
    ...
}

```

Lisp includes so-called *special forms*, which are functions that do not evaluate all arguments passed to them. For example, the `if` special form evaluates the test. If the test is true, then the then-expression is evaluated and returned. Otherwise the else-expression is evaluated and returned.

6.1 eval

(`eval expr`) evaluates an expression. This primitive is also called “unquote” since `expr` is typically a quoted Lisp expression:

```

L f_eval(L t,L e) { return eval(car(evlis(t,e)),e); }

```

All arguments to `eval` are evaluated with `evlis(t,e)` but `eval` only applies to one argument or the first argument when more than one is specified. Example: (`eval (quote (+ 1 2))`) gives 3.

6.2 quote

(`quote expr`) quotes an expression to keep it unevaluated. The Lisp parser also accepts `'expr`. Note that `expr` may be a list which means that the list and all of its elements remain unevaluated.

```

L f_quote(L t,L _) { return car(t); }

```

`quote` only applies to one argument or the first argument when more than one is specified, hence `car(t)` is returned. Example: (`quote (1 2 3)`) and `'(1 2 3)` give (1 2 3)

6.3 cons

(`cons expr1 expr2`) constructs a new pair (`expr1 . expr2`). Typically `expr2` is a list to construct a list with `expr1` at its head.

```

L f_cons(L t,L e) { return t = evlis(t,e),cons(car(t),car(cdr(t))); }

```

Example: (`cons 1 ()`) gives (1), (`cons 1 2`) gives the pair (1 . 2) and (`cons 1 (cons 2 ())`) gives the list (1 2).

6.4 car and cdr

(*car pair*) and (*cdr pair*) give the first and second element of a *pair*, respectively. This means that (*car list*) and (*cdr list*) give the element at the front of the *list* and the rest of the *list*, respectively.

```
L f_car(L t,L e) { return car(car(evlis(t,e))); }
L f_cdr(L t,L e) { return cdr(car(evlis(t,e))); }
```

6.5 Arithmetic

The four basic arithmetic operations are variadic functions:

```
L f_add(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n += car(t); return num(n); }
L f_sub(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n -= car(t); return num(n); }
L f_mul(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n *= car(t); return num(n); }
L f_div(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n /= car(t); return num(n); }
```

Note that the arithmetic functions expect at least one argument. They are undefined if no arguments are provided. Example: (+ 1 2 3 4) gives 10, (- 3 2) gives 1, and (- 3) gives 3, not -3. Some other Lisp may give -3, which can be implemented by checking if only one argument is passed to the function. I will leave it to you to change the implementation to support this feature.

6.6 int

(*int expr*) truncates *expr* to an integer.

```
L f_int(L t,L e) { L n = car(evlis(t,e)); return n-1e9 < 0 && n+1e9 > 0 ? (long)n : n; }
```

6.7 Comparison

(< *expr₁ expr₂*) and (eq? *expr₁ expr₂*) compare *expr₁* and *expr₂* to give #t when true or () when false; (pair? *expr*) returns #t when *expr* is a pair (a non-empty list).

```
L f_lt(L t,L e) { return t = evlis(t,e),car(t) - car(cdr(t)) < 0 ? tru : nil; }
L f_eq(L t,L e) { return t = evlis(t,e),equ(car(t),car(cdr(t))) ? tru : nil; }
L f_pair(L t,L e) { L x = car(evlis(t,e)); return T(x) == CONS ? tru : nil; }
```

Equality for pairs and lists is only true if the lists are the same objects in memory. Otherwise the pairs or lists are not equal, even when they contain the same elements. Example: (eq? 'a 'a) gives #t and (eq? '(a) '(a)) gives (). The less-than primitive compares numbers only and is always false for non-numeric arguments. Non-numeric types can be compared by comparing tags first and when equal comparing the ord values for example.

6.8 Logic

(not *expr*) gives #t when *expr* is () (the empty list) and () otherwise. (or *expr₁ expr₂ ... expr_n*) gives the value of the first *expr_i* that is not () (i.e. is true) and () otherwise if all *expr_i* are () (i.e. all are false). (and *expr₁ expr₂ ... expr_n*) gives the value of the last *expr_n* if all *expr_i* are not () (i.e. all are true) and () otherwise if any *expr_i* is () (i.e. is false).

```

L f_or(L t,L e) { L x = nil; while (!not(t) && not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_and(L t,L e) { L x = tru; while (!not(t) && !not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_not(L t,L e) { return not(car(evlis(t,e))) ? tru : nil; }

```

Only the first arguments to `or` and `and` are evaluated to determine the result. Example: `(and #t ())` gives `()` and `(and 1 2)` gives `2` since numbers are not `()`. Note that `and` returns the value of the *last expression* or `()`. Like the Lua programming language, the `or` and `and` combined produce an if-then-else of the form `(or (and test then) else)`. However, like Lua, this is not correct when *test* evaluates to true but *then* is nil (the `()` empty list.) In that case *else* is evaluated.

6.9 cond

`(cond (test1 expr1) (test2 expr2) ... (testn exprn))` evaluates the tests from the first to the last until *test_i* evaluates to true and then returns *expr_i*. At least one condition must be true.

```

L f_cond(L t,L e) {
  while (not(eval(car(car(t)),e))) t = cdr(t);
  return eval(car(cdr(car(t))),e);
}

```

Example: `(cond ((eq? 'a 'b) 1) ((< 2 1) 2) (#t 3))` gives `3`.

6.10 if

`(if test expr1 expr2)` evaluates and tests if *test* is true or false. If true (i.e. not `()`) then *expr₁* is evaluated and returned. Else *expr₂* is evaluated and returned.

```

L f_if(L t,L e) { return eval(car(cdr(not(eval(car(t),e)) ? cdr(t) : t)),e); }

```

Example: `(if (eq? 'a 'a) 1 2)` gives `1`.

6.11 let*

`(let* (var1 expr1) (var2 expr2) ... (varn exprn) expr)` defines a set of bindings¹¹ of variables in the scope of the body *expr* by evaluating each *expr_i* sequentially and associating *var_i* with the result.

```

I let(L x) { return !not(x) && !not(cdr(x)); }
L f_let*(L t,L e) {
  for (; let(t); t = cdr(t)) e = pair(car(car(t)),eval(car(cdr(car(t))),e),e);
  return eval(car(t),e);
}

```

The loop runs over the pairs in the `let*`. Each iteration extends the environment *e* with a pair that binds *var_i* (i.e. `car(car(t))`) to the value of *expr_i* (i.e. `eval(car(cdr(car(t))),e)`). Example: `(let* (a 3) (b (* a a)) (+ a b))` gives `12`.

¹¹Other Lisp require pairs `(var expr)` to be placed in a list such as `(let* ((a 3) (b (* a a))) (+ a b))`, but I prefer the simpler form shown here. Of course, you can change this implementation in any way you like.

6.12 lambda

(`lambda var expr`) and (`lambda (var1 var1 ... varn) expr`) create a closure, i.e. an anonymous function. The first form associates *var* with the list of arguments passed to the closure when the closure is applied. The second form associates each *var_i* with the corresponding argument passed to the closure when the closure is applied. Also the list dot may be used in the list of variables (`lambda (var1 . var2) expr`) to specify the remaining arguments to be passed as a list in *var₂*.

```
L f_lambda(L t,L e) { return closure(car(t),car(cdr(t)),e); }
```

Example: ((`lambda (x) (* x x)`) 3) gives 9 and ((`lambda (x y . args) args`) 1 2 3 4) gives (3 4).

6.13 define

(`define var expr`) globally defines *var* and associates it with the evaluated *expr*.

```
L f_define(L t,L e) { env = pair(car(t),eval(car(cdr(t))),e),env); return car(t); }
```

Globally defined functions may be (mutually) recursive. Example: after (`define pi 3.14`) the value of `pi` is 3.14, after (`define square (lambda (x) (* x x))`) the application (`square 3`) gives 9 and after (`define factorial (lambda (n) (if (< 1 n) (* n (factorial (- n 1))) 1))`) the application (`factorial 5`) gives 120.

We have not defined an `apply` primitive often found in Lisp implementations, because `apply` is not needed. To apply a function to a list of arguments (`f . args`) suffices, but only if `args` is a variable associated with a list of arguments¹². Otherwise, we shall use (`let* (args x) (f . args)`) to ensure `args` is a variable bound to the value of expression *x*.

7 Parsing Lisp Expressions

A Lisp tokenizer scans the input for tokens to return to the parser. A token is a single parenthesis, the quote character, or a white space delimited sequence of characters up to 39 characters long. The `scan` function populates `buf[40]` with the next token scanned from standard input. The token is stored as a 0-terminated string in `buf[]`:

```
char buf[40],see = 0;
void look() { int c = getchar(); see = c; if (c == EOF) exit(0); }
I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : see == c; }
char get() { char c = see; look(); return c; }
char scan() {
    int i = 0;
    while (seeing(' ')) look();
    if (seeing('(') || seeing(')') || seeing('\')) buf[i++] = get();
    else do buf[i++] = get(); while (i < 39 && !seeing('(') && !seeing(')') && !seeing(' '));
    return buf[i] = 0,*buf;
}
```

¹²The reason is that if we place a list after the dot like (`f . (g args)`), then the arguments passed to *f* are actually *g* and *args* as in (`f g args`), which is not what we want.

This implementation of `scan` does not support Lisp comments. I did this intentionally, since it is up to you to write your own implementation with the features that *you* want.

Lisp comments begin with a semicolon and end at the next line. To support comments, we can change the second line of the `scan` function to skip comments and continue scanning:

```
while (seeing(' ') || seeing(';')) if (get() == ';') while (!seeing('\n')) look();
```

Note that EOF is not checked. A neat little trick is to look for an EOF and then reopen standard input to read from the terminal:

```
void look() {
    int c = getchar();
    if (c == EOF) freopen("/dev/tty", "r", stdin), c = 0;
    see = c;
}
```

By changing `look` this way, we can now read a collection of Lisp definitions from a file before the interactive session starts, by using the Linux/Unix `cat` utility:

```
bash$ cat common.lisp list.lisp math.lisp | ./tinylisp
```

This sends the `common.lisp`, `list.lisp` and `math.lisp` files (see Appendix E) to the interpreter. After EOF of `cat`, the Lisp interpreter is ready to accept input again, this time from the terminal. *Beware that the number of cells `N` must be increased to import this many definitions!*

To parse Lisp expressions we employ a *recursive-descent parsing* technique. The `Read` function¹³ returns a Lisp expression parsed from the input by invoking the `scan` and `parse` functions:

```
L Read() { return scan(), parse(); }
```

where the `parse` function parses a list, a quoted expression, or an atomic expression (a symbol or a number):

```
L parse() { return *buf == '(' ? list() : *buf == '\'' ? quote() : atomic(); }
```

The `list` function recursively parses and constructs a list up to the closing parenthesis `)`. In addition, a dot in a list creates a pair:

```
L list() {
    L x;
    return scan() == ')' ? nil :
        !strcmp(buf, ".") ? (x = Read(), scan(), x) :
        (x = parse(), cons(x, list()));
}
```

¹³Not to be confused with the `unistd.h` `read` function. To avoid link failures we use `Read`.

Note that `x = parse()` must be called before the parsed value is used in `cons(x,list())`, because argument evaluation order in C is undefined, i.e. not necessarily left-to-right. You may have noticed by now that I use the C comma operator a lot, including for this specific purpose and to keep the code compact. The C comma operator has a cousin in Lisp (`begin expr1 expr2 ... exprn`) sometimes called `progn` that returns the value of the last expression `exprn`. So its use is sanctioned by Lisp to sequence expressions as statements with *side effects*¹⁴.

Parsing the list `(x y z)` for example, results in the list construction `(cons x (cons y (cons z nil)))` and parsing `(x y . args)` results in the construction `(cons x (cons y args))`. Parsing a quoted expression `'expr` produces `(quote expr)`:

```
L quote() { return cons(atom("quote"),cons(Read(),nil)); }
```

Parsing an atomic expression produces a number if the token is numeric and an atom otherwise:

```
L atomic() { L n; I i; return sscanf(buf,"%lg%n",&n,&i) > 0 && !buf[i] ? n : atom(buf); }
```

A token must be numeric to convert it to a number. If it is not, then an atom with the specified tokenized name is returned. Note that `sscanf` accepts `inf` and `-inf` as numbers and hexadecimal `0x...h`, all of which we get it for free. A *NaN* corresponds to the `ERR` atom with zero offset into the heap. Therefore, the *NaN* value produced with the sum `(+ inf -inf)` is reported as `ERR`.

The PC-G850 requires function `atomic` to be modified as follows:

```
L atomic() {
  L n; int i = strlen(buf);
  return isdigit(buf[*buf == '-']) && sscanf(buf,"%lg%n",&n,&i) && !buf[i] ? n : atom(buf);
}
```

where `i` must be initialized to the length of the buffer passed to `sscanf`. Because `sscanf` on the PC-G850 simply returns 0 for incomplete numeric forms such as a single character `-`, we check if the token begins with a digit after an optional minus sign.

8 Printing Lisp Expressions

Displaying Lisp expression requires a few lines of code. This code should be self-explanatory:

```
void print(L x) {
  if (T(x) == NIL) printf("()");
  else if (T(x) == ATOM) printf("%s",A+ord(x));
  else if (T(x) == PRIM) printf("<%s>",prim[ord(x)].s);
  else if (T(x) == CONS) printlist(x);
  else if (T(x) == CLOS) printf("{%u}",ord(x));
  else printf("%.10lg",x);
}
```

¹⁴Functions with side effects affect the state of the machine outside of their arguments and locals, such as through assignments to non-local variables and by performing IO operations. Pure functional programming bans them. For C and Lisp we must properly sequence functions with side effects to avoid undefined behavior.

Function `printlist` iterates over the list to display its elements in order. It prints a dot for the last cons pair if the list does not end in `nil`:

```
void printlist(L t) {
    for (putchar('('); ; putchar(' ')) {
        print(car(t));
        if (not(t = cdr(t))) break;
        if (T(t) != CONS) { printf(" . "); print(t); break; }
    }
    putchar(')');
}
```

Note that `not(t = cdr(t))` changes `t` to the next list pair. Then, if `t` is `nil` we break from the loop. Otherwise, if the next `t` is not a cons pair, then we display a dot followed by the value of `t`. The dot visually separates the pair's values. The dot is also used to construct pairs, see Section 7.

9 Garbage Collection

To keep our Lisp interpreter code tiny, we want to implement a very simple form of garbage collection to delete all temporary cells from the stack. We should preserve all globally-defined names and functions listed in `env`. To delete all temporary cells and keep `env` intact, it suffices to restore the stack pointer to the point on the stack where the free space begins, which is right below the global environment `env` cell on the stack:

```
void gc() { sp = ord(env); }
```

Why does this work? After the last `env = pair(name, expr, env)` call was made to define `name` globally, we know for sure that `expr` is already stored higher up in the stack in cells above the last `env` pair on the stack. These cells are not removed by `gc` when we set `sp = ord(env)`.

One caveat of this approach is that we cannot support interactive use of the Lisp special forms `setq` (modifies an association in an environment, Section 11.3), `set-car!` and `set-cdr!` (overwrites the `car` or `cdr` of a pair, Section 11.4), since these may change previously-defined expressions in the global environment. If the modified global environment references temporary lists, then `gc` corrupts the global environment by removing these lists. We can support `setq`, `set-car!` and `set-cdr!` if we only assign atomic values to globals. Fortunately, locals in our Lisp code are always assignable.

This simple one-line garbage collector does not remove unused symbols. Temporary atoms on the heap are kept. A minor addition suffices to remove unused atoms from the heap. We only need to find the max heap reference among the used `ATOM`-tagged cells and adjust `hp` accordingly:

```
void gc() {
    I i = sp = ord(env);
    for (hp = 0; i < N; ++i) if (T(cell[i]) == ATOM && ord(cell[i]) > hp) hp = ord(cell[i]);
    hp += strlen(A+hp)+1;
}
```

It's simple and effective. But doing a `gc` after running a program won't suffice to avoid running out of memory for long-running programs that create lots of lists. For that, we need a real garbage collector¹⁵.

¹⁵See the project's GC directory if you are interested in *reference counting* and simple *mark-sweep* garbage collection.

10 The Read-Eval-Print Loop

After the `main` program initialized the static variables `nil`, `tru`, and `err` (see Section 4) and populated the environment with `#t` and other primitives (see Section 6), the `main` program executes the so-called Lisp *read-eval-print loop* (REPL):

```
int main() {
    ...
    while (1) { printf("\n%u>",sp-hp/8); print(eval(Read(),env)); gc(); }
}
```

The prompt in the REPL displays the number of cells freely available, i.e. the space between the heap pointer `hp` and stack pointer `sp`. Note that `hp` points to bytes and `sp` points to 8-byte floats, so `hp` is scaled down by a factor 8. Garbage collection is performed in the REPL after the results are displayed.

This completes Lisp in 99 lines¹⁶ of C. See Appendix A and B for the complete listings with NaN and BCD boxing, respectively.

11 Additional Lisp Primitives

The following Lisp primitives are not included in the 99 line C program, since these are not absolutely required to write Lisp programs.

11.1 `assoc` and `env`

(`assoc var environment`) gives the expression associated with `var` in the specified *environment*. (`env`) returns the current environment in which (`env`) is evaluated.

```
L f_assoc(L t,L e) { t = evlis(t,e); return assoc(car(t),car(cdr(t))); }
L f_env(L _,L e) { return e; }
... prim[] = { ... {"assoc",f_assoc},{"env",f_env} ... };
```

Note that *var* should be quoted when passed to `assoc` since its arguments are evaluated first. Example: (`assoc 'b '((a . 1) (b . 2) (c . 3))`) gives 2.

11.2 `let` and `letrec`

The `let` special form¹⁷ is similar to the `let*` special form, but evaluates all expressions first in the outer scope before binding the values to the variables.

```
L f_let(L t,L e) {
    L d = e;
    for (; let(t); t = cdr(t)) d = pair(car(car(t)),eval(car(cdr(car(t))),e),d);
    return eval(car(t),d);
}
... prim[] = { ... {"let",f_let} ... };
```

¹⁶That is, a Lisp-like functional style of structured C. Lines are 55 columns wide on average and never wider than 120 columns for convenient editing.

¹⁷This `let` syntax differs from other Lisp: like our `let*` we don't need to put all the *var-expr* pairs in a list. Change the syntax as you like.

The `letrec*` special form is similar to the `let*` special form, but allows for self-recursion in lambda closures and a local variable name may also appear in the value of a subsequent name-value pair. For example:

```
> (letrec* (f (lambda (n) (if (< 1 n) (* n (f (- n 1))) 1))) (f 5))
120
```

The trick is that we add variable-to-`nil` bindings to the environment `e`, then override the `nil` cell on the stack at `cell[ord(car(e))]` with the expression evaluated with the updated `e`:

```
L f_letreca(L t,L e) {
  for (; let(t); t = cdr(t)) e = pair(car(car(t)),nil,e),cell[ord(car(e))] = eval(car(cdr(car(t))),e);
  return eval(car(t),e);
}
... prim[] = { ... {"letrec*",f_letreca} ... };
```

The `letrec` special form is similar to the `letrec*` special form, but allows for mutually-callable (recursive) lambda closures. The trick is to construct the list of local bindings `d` first, then iterate again over the bindings to evaluate the expressions with the updated environment `e = d`:

```
L f_letrec(L t,L e) {
  L s,d,*p;
  for (s = t,d = e,p = &d; let(s); s = cdr(s),p = &cell[ord(*p)]) *p = pair(car(car(s)),nil,e);
  for (e = d; let(t); t = cdr(t),d = cdr(d)) cell[ord(car(d))] = eval(car(cdr(car(t))),e);
  return eval(car(t),e);
}
... prim[] = { ... {"letrec",f_letrec} ... };
```

11.3 `setq`

The `setq` special form sets the value of a variable as a side-effect with `(setq var expr)`:

```
L f_setq(L t,L e) {
  L d = e,v = car(t),x = eval(car(cdr(t)),e);
  while (T(d) == CONS && !equ(v,car(car(d)))) d = cdr(d);
  return T(d) == CONS ? cell[ord(car(d))] = x : err;
}
... prim[] = { ... {"setq",f_setq} ... };
```

This function is dangerous, because garbage collection after `setq` may corrupt the stack if the new value assigned to a global variable is a temporary list (all interactively constructed lists are temporary). On the other hand, atomic values are always safe to assign and `setq` is safe to use to assign local variables in the scope of a `lambda` and a `let`.

11.4 `set-car!` and `set-cdr!`

`(set-car! pair expr)` sets the value of the `car` cell of a cons `pair` to `expr` as a side-effect, and `(set-cdr! pair expr)` sets the value of the `cdr` cell of a cons `pair` to `expr` as a side-effect.

```

L f_setcar(L t,L e) {
    L p = car(t = evlis(t,e));
    return (T(p) == CONS) ? cell[ord(p)+1] = car(cdr(t)) : err;
}
L f_setcdr(L t,L e) {
    L p = car(t = evlis(t,e));
    return (T(p) == CONS) ? cell[ord(p)] = car(cdr(t)) : err;
}
... prim[] = { ... {"set-car!",f_setcar},{"set-cdr!",f_setcdr} ... };

```

Like `setq`, these functions are dangerous. `set-car!` and `set-cdr!` return the value of *expr*.

11.5 macro

Macros allow Lisp to be syntactically extended. A **macro** is similar to a **lambda**, except that its arguments are not evaluated when the macro is applied. Typically a macro constructs Lisp code when applied, thereby *expanding* and evaluating the Lisp code in place.

To add the (macro *variables expression*) primitive is easy. It only requires a few lines of code to define a new MACR tag, add new the functions `macro`, `f_macro` and `expand`, and make some minor changes to the existing functions `car`, `cdr` and `apply`. First we add a new tag MACR:

```

/** with NaN Boxing */
I ATOM=0x7ff8,PRIM=0x7ff9,CONS=0x7ffa,CLOS=0x7ffb,MACR=0x7ffc,NIL=0x7ffd;
/** with BCD boxing */
I ATOM=32,PRIM=48,CONS=64,CLOS=80,MACR=96,NIL=112;

```

We must modify the `car` and `cdr` functions to apply to MACR-tagged floats, which are essentially cons pairs containing the list of *variables* of the macro as `car` cell and *expression* as the `cdr` cell:

```

L car(L p) { return T(p) == CONS || T(p) == CLOS || T(p) == MACR ? cell[ord(p)+1] : err; }
L cdr(L p) { return T(p) == CONS || T(p) == CLOS || T(p) == MACR ? cell[ord(p)] : err; }

```

We add a constructor `macro` and the corresponding Lisp primitive `f_macro`:

```

L macro(L v,L x) { return box(MACR,ord(cons(v,x))); }
L f_macro(L t,L e) { return macro(car(t),car(cdr(t))); }
... prim[] = { ... {"macro",f_macro} ... };
...
void print(L x) {
    ...
    else if (T(x) == MACR) printf("[%u]",ord(x));
}

```

Application of macros is similar to lambdas, but they expand by a new `expand` function:

```

L expand(L f,L t,L e) { return eval(eval(cdr(f),bind(car(f),t,env)),e); }
L apply(L f,L t,L e) {
    return T(f) == PRIM ? prim[ord(f)].f(t,e) :
        T(f) == CLOS ? reduce(f,t,e) :
        T(f) == MACR ? expand(f,t,e) :
        err;
}

```

The macro is evaluated in the global environment `env`. This typically constructs Lisp code that is then evaluated in the current environment `e`.

Example: the Lisp *delayed evaluation* primitives `delay` and `force` implemented as a macro:

```
> (define list (lambda (args) args))
> (define delay (macro (x) (list 'lambda () x)))
> (define force (lambda (f) (f)))
```

The `list` function returns a list of evaluated arguments passed to it. This is quicker to construct a list without having to use a series of nested `cons`. The `delay` macro offers a simple form of *lazy evaluation* or *call by need* of arguments, by passing them unevaluated to a function together with their environment as a closure called a *promise*. Hence, `(list 'lambda () x)` constructs the Lisp code `(lambda () x)` where `x` is the unevaluated argument passed to `delay`. The closure of this `lambda` includes the proper environment in which `x` should be evaluated, thus respecting its static scope. The `force` function evaluates a promise:

```
> (force (delay (+ 1 2)))
3
```

Note that `force` executes the closure returned by the macro, which is simply the same as `((delay (+ 1 2)))`. More information and examples of `delay` and `force` can be found in Lisp textbooks and manuals.

With macros we can also define `defun` to define functions more easily without a `lambda`:

```
> (define defun (macro (f v x) (list 'define f (list 'lambda v x))))
> (defun square (n) (* n n))
> (square 3)
9
```

Macros are powerful, but defining them typically requires a lot of quoting and `list` usage to construct Lisp code with a macro. With the Lisp *backquote* (```) and *comma* (`,`) operators this effort can be simplified. For example, with backquote and comma operators, the `defun` macro is:

```
> (define defun (macro (f v x) `(define ,f (lambda ,v ,x))))
```

The backquote operator inserts `list` into the head of each (nested) list and quotes everything else in the lists, except when a comma operator is used. The comma operator escapes quoting (“*down quotes*”). Therefore, in the example above, the `f`, `v` and `x` macro arguments escape quoting and will evaluate to their values. Note that without commas, the result of backquoting a list is essentially the same as quoting the list (but quoting the list is more efficient.)

To understand the result of backquoting, the following expressions are semantically equivalent:

```
`(a ,b (c ,d) (e f))
(list 'a b (list 'c d) '(e f))
(cons 'a (cons b (cons (cons 'c (cons d ())) '(e f))))
(cons (quote a) (cons b (cons (cons (quote c) (cons d ())) (quote (e f))))))
```

In all cases, `b` and `d` are replaced by their values `bv` and `dv` respectively: `(a bv (c dv) (e f))`.

To add the backquote syntax to our Lisp interpreter, we update the `scan` and `parse` functions. The `scan` function should also accept the backquote and comma operators as tokens when seeing (`'``) or seeing (`','`) in the input, so we include these as two additional conditions:

```

char scan() {
    ...
    if (seeing('(') ||
        seeing(')') ||
        seeing('\\') ||
        seeing('`') ||
        seeing(',')) buf[i++] = get();
    ...
}

```

The `parse` function should also parse backquoted expressions, `scan` and `tick` what follows:

```

L parse() {
    return *buf == '(' ? list() :
           *buf == '\\' ? quote() :
           *buf == '`' ? scan(),tick() :
           atomic();
}

```

When a backquote ``` is parsed in the input, the `tick` function parses the applicable expression and constructs a translated representation using the `list` and `quote` Lisp functions:

```

L ticklist();
L tick() {
    return *buf == '(' ? cons(atom("list"),ticklist()) :
           *buf == ',' ? Read() :
           cons(atom("quote"),cons(parse(),nil));
}
L ticklist() { L x; return scan() == ')' ? nil : x = tick(),cons(x,ticklist()); }

```

For example, ``(a ,b (c ,d) (e f))` is parsed as `(list (quote a) b (list (quote c) d) (list (quote e) (quote f)))`, which when evaluated yields `(a bv (c dv) (e f))`. Note that the `list` Lisp function is not a built-in primitive, but defined externally in `common.lisp`, see Appendix E.

11.6 read and print

Our tiny Lisp implementation includes `Read` and `print` functions. Let's add them as Lisp primitives:

```

L f_read(L t,L e) { L x; char c = see; see = 0; x = Read(); see = c; return x; }
L f_print(L t,L e) {
    for (t = evlis(t,e); !not(t); t = cdr(t)) print(car(t));
    return nil;
}
L f_println(L t,L e) { f_print(t,e); putchar('\n'); return nil; }
... prim[] = { ... {"read",f_read},{"print",f_print},{"println",f_println} ... };

```

For example, `(read)` gives the Lisp expression typed in (unevaluated), `(print 'hello 123)` displays `hello123`, and `(println '(hello world))` displays `(hello world)`.

With a few more lines of C code, you can add your own Lisp primitives for a more complete IO implementation. You could store open file `FILE*` streams in an array to support `open` and `close` primitives on multiple streams in Lisp. Then index this array by a float number in Lisp to obtain the `FILE*` for an internal IO operation. Some Lisp implementations include a special *port* type for this, which is basically a `FILE*`.

12 Adding Readline with History

Command shells such as `bash` use GNU `readline` for interactive input. `Readline` plays nice with interactive input and provides a history mechanism to let us recall previous input. The `readline` library will be a nice addition to our Lisp. While we are at it, we might as well read a Lisp initialization file `common.lisp` with common Lisp definitions before the interactive prompt. First we need to include the usual C headers. There are two for `readline`:

```
#include <readline/readline.h>
#include <readline/history.h>
```

Besides the `buf` and `see` globals, we also need a `ptr` pointing at the current character in the `line` string returned by `readline`. We also add a prompt string `ps` to display the Lisp prompt and an `in` pointer to the open file with input, i.e. to read `common.lisp` when opened successfully:

```
char buf[40], see = 0, *ptr = "", *line = NULL, ps[20];
FILE *in = NULL;
```

Function `look` is modified to read a character from `in` when `in` is not `NULL`, or to read a line of input with `readline`. Function `seeing` is modified to match `\n` also with the end-of-string zero:

```
void look() {
    if (in) {
        int c = getc(in);
        see = c;
        if (c != EOF) return;
        fclose(in);
        in = NULL;
        see = 0;
    }
    if (!see) {
        if (line) free(line);
        while (!(ptr = line = readline(ps))) freopen("/dev/tty", "r", stdin);
        add_history(line);
        strcpy(ps, "? ");
    }
    see = *ptr++;
}
I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : (c == '\n' && !see) || see == c; }
```

To read `common.lisp` or a file specified on the command line, let's modify `main` as follows:

```
int main(int argc, char **argv) {
    ...
    in = fopen((argc > 1 ? argv[1] : "common.lisp"), "r");
    using_history();
    while (1) { putchar('\n'); snprintf(ps, 20, "%u>", sp-hp/8); print(eval(Read(), env)); gc(); }
}
```

Note that the modified REPL populates the prompt string `ps` with `snprintf` to display the number of remaining free cells. The prompt string is set to `?` in the `look` function when the interactive input is not yet complete. The `libreadline` library must be linked with our updated source:

```
bash$ cc -o tinylisp-opt.c -lreadline
```

Now you're all set to enjoy enhanced interactive input! Moreover, since we read any file when `in` is points to a `FILE`, let's use it to define a new `load` function that loads a Lisp file:

```
L f_load(L t,L *e) { L x = car(t); return !in && T(x) == ATOM && (in = fopen(A+ord(x),"r")) ? x : err(5,x); }
... prim[] = { ... {"load",f_load}, ... };
```

If a file is not already open and the first argument of `load` is an atom, then we open the file with that name for reading. Otherwise, we return `err(5,x)` (cannot open). For example:

```
(load some.lisp)
```

13 Tracing Lisp

Everything that is happening behind the scenes in our Lisp interpreter is essentially just the traversal of lists as code to return numbers, atoms, closures and lists as data. But what does that actually look like? Well, let's find out by tracing the evaluation steps in our Lisp. When we display every evaluation step performed by `eval` we see exactly what is happening in detail:

```
bash$ tinylisp
930>(define sq (lambda (n) (* n n)))
920 define => <define>
920 lambda => <lambda>
916 (lambda (n) (* n n)) => {916}
912 (define sq (lambda (n) (* n n))) => sq
sq
901>(sq 3)
908 sq => {916}
908 3 => 3
908 () => ()
902 * => <*>
902 n => 3
902 n => 3
902 () => ()
898 (* n n) => 9
898 (sq 3) => 9
9
901>
```

A trace displays the Lisp code and data before `eval` followed by the Lisp code and data after `eval`. To implement tracing, we rename `eval` to `step` and add a new `eval` function to display the trace:

```
L step(L x,L e) { ... } /* this is the old eval() function renamed to step() */
void print(L);          /* function prototype moved up since we need print() */
L eval(L x,L e) {
    L y = step(x,e);
    printf("%u ",sp); print(x); printf(" => "); print(y);
    while (getchar() >= ' ') continue;
    return y;
}
```

The `while`-loop in `eval` waits until the RETURN key is pressed (or any special key or EOF). This always displays the trace of your Lisp. With a few more lines of C, you could make tracing an option enabled with a new (`trace state`) Lisp function that takes `state` 0, 1 or 2 to set a global variable. When this variable is set to 1 or 2, the trace output is displayed in `eval` and when 2 it also waits for a keypress. With this addition to your Lisp, you can now step through your code on demand. In addition to tracing, you could also dump the stack. For example, when the user types a `d` for dump followed by RETURN.

14 Adding Error Handling and Exceptions

Error handling is practically non-existent in our Lisp at this point. It is an error to take the `car` or `cdr` of a non-pair, to use the value of an undefined symbol, or to try to apply a non-function to arguments. These three error conditions produce the `ERR` value in our Lisp. We could extensively debug our code by tracing (see Section 13) to find all possible bugs and add `err?` checks to our Lisp functions to catch problems (see Appendix E for a definition of `err?`). Analyzing your Lisp code for correctness and debugging your Lisp functions is critical, but adding `err?` calls to Lisp code is cumbersome. Wouldn't it be nice to have a mechanism to catch exceptions?

There are two mechanisms to implement exception handling in our Lisp: in C using `setjmp` or in C++ with `try-catch` if we rename and compile our project in C++.

14.1 C `setjmp`

The `setjmp(jmp_buf jb)` function saves its *calling environment* in `jb` and returns zero. The corresponding `longjmp(jmp_buf jb, int n)` function restores the calling environment `jb` saved by the most recent `setjmp(jb)` and passes `n` to the return value of this `setjmp`. What basically happens is that `longjmp` deletes all active function calls since the last `setjmp` and immediately returns to `setjmp`. This mechanism is somewhat similar to C++ exceptions with `longjmp` acting like `throw` and `setjmp` acting like `try-catch`. As with C++ exceptions, we must be careful to finalize unfinished business before throwing exceptions or add exception handlers to finalize unfinished business, re-throwing the exception when applicable. Unfinished business include open files that must be closed and memory allocations that must be freed. Fortunately, we have none of these issues to worry about¹⁸ in our Lisp interpreter.

After including `setjmp` in our interpreter, we define a global calling environment `jmp_buf jb` and a new `err` function to “throw” errors by invoking `longjmp(jb, n)` for nonzero error codes `n`:

```
#include <setjmp.h>
...
jmp_buf jb;
L err(int i) { longjmp(jb,i); }
```

Note that the new `err` function doesn't return a value (you should be able to figure out why it makes no sense to return a value.) With the new `err` function, we change the uses of `err` in our implementation to throw different error codes:

¹⁸Unless you've added file open and close to your Lisp interpreter's primitives. In that case, it helps to keep a table of open file descriptors and close them all when an exception occurs.

```

L car(L p) { return (T(p)&~(CONS^CLOS)) == CONS ? cell[ord(p)+1] : err(1); }
L cdr(L p) { return (T(p)&~(CONS^CLOS)) == CONS ? cell[ord(p)] : err(1); }
...
L assoc(L v,L e) {
    while (T(e) == CONS && !equ(v,car(car(e)))) e = cdr(e);
    return T(e) == CONS ? cdr(car(e)) : err(2);
}
...
L apply(L f,L t,L e) {
    return T(f) == PRIM ? prim[ord(f)].f(t,e) :
           T(f) == CLOS ? reduce(f,t,e) :
           err(3);
}

```

We still want to call `atom("ERR")` in `main` to populate the heap with a `ERR` atom first, because its zero `ord` is associated with a quiet NaN. We invoke `setjmp(jb)` in `main` to initialize `jb` and catch errors in the REPL, which is when `setjmp` returns a nonzero value:

```

int main() {
    I i; printf("tinylisp");
    nil = box(NIL,0); atom("ERR"); tru = atom("#t"); env = pair(tru,tru,nil);
    ...
    if ((i = setjmp(jb)) != 0) printf("ERR %d",i);
    while (1) { gc(); putchar('\n'); snprintf(ps,20,"%u>",sp-hp/8); print(eval(Read(),env)); }
}

```

The `err` assignment is replaced with just `atom("ERR")` as noted. We also moved `gc()` up to the front in the REPL. Now all errors are caught by `setjmp` and reported:

```

930>(car 3)
ERR 1
930>foo
ERR 2
930>(1 2)
ERR 3

```

When tracing is added to our interpreter (see the previous section), we can also display the cause of an error by calling `err(1,p)`, `err(2,v)`, `err(3,f)` and update the `err` function as follows:

```

L err(I i,L x) {
    if (tr) { printf("\n\e[31;1mERR %u: ",i); print(x); printf("\e[m\n"); }
    longjmp(jb,i);
}

```

Another improvement is to replace `abort()` with `err(4,nil)` when we run out of memory. To make this useful, we should update the condition in function `cons` to `if (hp+16 > sp<<3) err(4,nil)` to add sufficient slack of at least two cells (16 bytes) to prevent destroying the atom heap.

In addition to our new and practical exception handling mechanism, there are several ways we can implement `catch` and `throw` Lisp primitives. The following is a simplified version that allows you to `(throw n)` an error code $n > 0$ that is caught as an `(ERR . n)` pair returned by `(catch expression)` whenever a `(throw n)` is explicitly invoked or when an `err(n,x)` occurs in the interpreter when `expression` is evaluated. The out-of-memory (4) and stopped (6) errors should not be caught and are re-raised to propagate down to the REPL:

```

L f_catch(L t,L e) {
  L x; int i;
  jmp_buf savedjb;
  memcpy(savedjb,jb,sizeof(jb));
  if ((i = setjmp(jb)) == 0) x = eval(car(t),*e);
  memcpy(jb,savedjb,sizeof(jb));
  return i == 0 ? x : i == 4 || i == 6 ? err(i,nil) : cons(atom("ERR"),i);
}
L f_throw(L t,L e) { longjmp(jb,(int)num(car(t))); }
... prim[] = { ... {"catch",f_catch},{"throw",f_throw} ... };

```

where **throw** is a special form that does not evaluate its argument, which must be a constant integer. With some more C code you could pass along the cell index of a cons pair on the stack instead of error codes. That means you could throw lists as errors with more information. However, care must be taken to distinguish internal error codes from explicit exceptions thrown.

14.2 Catching SIGINT to Stop Running Programs

When running a Lisp program that doesn't terminate for some reason, either intentionally or not, then we would like to interrupt the program and return to the REPL. To do this we need to define a **SIGINT** signal handler. The handler is invoked asynchronously when CTRL-C is pressed. When this happens we can simply return to the REPL using **err(6,nil)** that calls **longjmp(jb,6)** with 6 being the error number to report that the program was stopped. This returns control to the last **setjmp** executed. However, when pressing CTRL-C at the REPL, we may want to terminate the Lisp interpreter with an **abort()** to not return to the REPL:

```

#include <signal.h>
void stop(int i) { if (line) err(6,nil); else abort(); }
...
int main(int argc, char **argv) {
  ...
  if ((i = setjmp(jb)) > 0) printf("ERR %u",i);
  signal(SIGINT, stop);
  while (1) { gc(); putchar('\n'); snprintf(ps,20,"%u>",sp-hp/8); print(eval(Read(),env)); }
}

```

We check if the input **line** string to parse returned by **readline** is valid to determine if the programming is running or waiting on user input in which case it is **NULL** and we **abort**. We call **signal** after **setjmp** to make sure **jb** is initialized. We moved **gc()** up to the front of the loop so its executed right after an exception. The **look** function is changed to set **line** to **NULL** when freed:

```

void look() {
  ...
  if (!see) {
    if (line) { ptr = line; line = NULL; free(ptr); }
    while (!(ptr = line = readline(ps))) freopen("/dev/tty","r",stdin);
  }
  ...
}

```

We are very cautious to set **line** to **NULL** *before* we free its string data. If we would free first and then set **line** to **NULL** then there is an infinitesimal chance that CTRL-C could interrupt the

program *after* the `free` and *before* `line` is set to `NULL`. That leads to a *use-after-free*, including a *double free* of the `line` string later.

14.3 Using C++ Exceptions

Compiling our Lisp interpreter in C++ requires almost no change (just simple stuff, e.g. renaming `not` to `Not`). With C++ we can use `try-catch` and `throw` in the REPL:

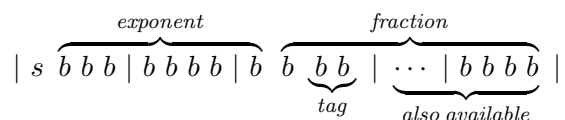
```
L err(int i) { throw(i); }
...
int main() {
    ...
    while (1) {
        printf("\n%u>", sp-hp/8);
        try { print(eval(Read(), env)); }
        catch (int i) { printf("ERR %d", i); }
        gc();
    }
}
```

We also change the `car`, `cdr`, `assoc` and `apply` functions as shown in the previous section. Lisp `catch` and `throw` primitives can be defined with the usual C++ `try-catch-throw`.

15 Downsizing Lisp to Single Floating Point Precision

Our Lisp supports double precision floating point values and operations on them without compromise. We can halve the memory use of our Lisp by using single precision floating point instead. This is a small compromise to make to reduce memory consumption if memory comes at a premium or if the machine does not support double floating point precision. Despite downsizing our Lisp, it won't be handicapped to run toy Lisp examples only. In fact, it can handle up to $N = 2^{20}/4 = 262,144$ cells (1,048,576 bytes) to evaluate Lisp.

A single precision NaN allows up to 22 bits to be arbitrarily used to stuff any information we want into a single precision NaN:



All exponent bits must be set for NaN. This time we use the sign bit *s* together with our *tag* since we need 3 bits for tagging. Therefore, `NIL` has tag `0xffff` with the sign bit set as part of the tag. We also want to maximize the use of the 23 bit fraction. Of these 23 bits the most significant bit must be set for quiet NaN. We use 2 bits for the tag and are left with 20 bits to store an integer “ordinal” value to refer to atoms on the heap, to refer to primitives in the `prim[]` array, and to refer to cons cells and closure cells on the stack.

Only a few minor changes are required to the code to downsize our Lisp to single precision floating point with 4-byte stack cells. We adjust the NaN-boxing tags and reduce the ordinals to 20 bit by changing the NaN-boxing-related functions:

```

#define L float
#define T(x) *(uint32_t*)&x>>20
...
#define N 1024 /* N should not exceed 262144 = 2^20/4 cells = 1048576 bytes */
I hp=0,sp=N,ATOM=0x7fc,PRIM=0x7fd,CONS=0x7fe,CLOS=0x7ff,NIL=0xfff;
...
L box(I t,I i) { L x; *(uint32_t*)&x = (uint32_t)t<<20|i; return x; }
I ord(L x) { return *(uint32_t*)&x & 0xffff; }
L num(L n) { return n; }
I equ(L x,L y) { return *(uint32_t*)&x == *(uint32_t*)&y; }

```

The float-addressing stack pointer `sp` should be scaled by 4 instead of 8 to compare to the byte-addressing heap pointer `hp` to check if they meet. Therefore, the `atom` and `cons` functions should be modified to use `sp<<2`:

```

L atom(const char *s) {
    ...
    if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<2) abort();
    ...
}
L cons(L x,L y) {
    ...
    if (hp+8 > sp<<2) abort(); /* or err(4); */
    ...
}

```

Likewise, the prompt displayed in the REPL is changed to use `sp-hp/4`:

```

int main() {
    ...
    while (1) { printf("\n%u>",sp-hp/4); print(eval(Read(),env)); gc(); }
}

```

Furthermore, we tweak the `atomic` and `print` functions to correctly parse and print single precision floats, respectively:

```

L atomic() { L n; int i; return sscanf(buf,"%g%n",&n,&i) > 0 && !buf[i] ? n : atom(buf); }
...
void print(L x) {
    ...
    else printf("%g",x);
}

```

Finally, we should also change `f_int` to truncate floats to 32 bit integers:

```

L f_int(L t,L e) { L n = car(evlis(t,e)); return n < 1e7 && n > -1e7 ? (int32_t)n: n; }

```

16 Optimizing the Interpreter

Understanding Lisp and Lisp evaluation does not require a deep understanding of the optimizations we may want to apply to make our interpreter run faster and use less memory. Optimization is generally nice, but a more critical goal is to optimize tail-calls performed by Lisp functions. Tail-call optimization effectively permits recursion in Lisp without the risk of running out of stack space. To achieve this, we start by optimizing recursion away in our interpreter by replacing recursive calls with loops when possible, then replace `eval` with a tail-call optimized iterative version.

16.1 Replacing Recursion with Loops

The `evlis` C function is the first target we pick for optimization. It calls itself recursively to construct a list of evaluated expressions. The `evlis` function does this by consing the evaluated `car` expression of the list `t` to the `evlis` rest of the list `t` in `(cons(eval(car(t)),e),evlis(cdr(t),e))`. This is inefficient. Worse, the C compiler can't tail-call optimize this C code to avoid the calling overhead and the stack growth associated with it.

Effective C programming with pointers can be exploited to replace recursion in `evlis` by iteration using a pointer `p` that points to the last `cdr` cell of the list we are constructing in `evlis`. To construct the list iteratively, we just need to replace the last `nil` cell of the list that `*p` points to and replace it by a new `cons`, then update `p` to point to the `nil` of that `cons` with `p = cell+sp`:

```
L evlis(L t,L e) {
    L s,*p;
    for (s = nil,p = &s; T(t) == CONS; p = cell+sp,t = cdr(t)) *p = cons(eval(car(t)),e,nil);
    if (T(t) == ATOM) *p = assoc(t,e);
    return s;
}
```

Instead of `p = cell+sp` which points to the last cell created on the stack that happens to be the `cdr` cell of the last `cons`, we can also use `p = &cell[ord(*p)]` or simply `p = cell+ord(*p)` as an alternative. Initially, `p` points to `s` with `s = nil` as the first step to get the list construction started. Once the list is complete, we return `s` as the result of `evlis`. Eventually the list `t` runs out in a non-CONS value. This is typically a `T(t) == NIL`, but can instead be a symbol when the function application list uses the dot operator as in `(f x . args)` for example. The value of the dotted `args` is appended to the list we construct. This is performed by `if (T(t) == ATOM) *p = assoc(t,e)` in the optimized version of `evlis`. Another function to optimize is `list` to parse lists:

```
L list() {
    L t,*p;
    for (t = nil,p = &t; ; *p = cons(parse(),nil),p = cell+sp) {
        if (scan() == ')') return t;
        if (*buf == '.' && !buf[1]) return *p = Read(),scan(),t;
    }
}
```

This optimization mirrors the `evlis` optimization by assigning `*p = cons(parse(),nil)` and updating the pointer `p = cell+sp`. Likewise, we optimize `tick` to parse backquotes, remove the `ticklist` function, support dot-operators, such as ``(x y . zs)` translates to `(append (list 'x 'y) 'zs)` that evaluates back to `(x y . zs)`, and quoted expressions in backquotes are quoted:

```

L quote(L x) { return cons(atom("quote"),cons(x,nil)); }          /* returns (quote x) */
L tick() {
  L t,*p;
  if (*buf == ',') return Read();
  if (*buf == '\\') return scan(),cons(atom("list"),cons(quote(atom("quote")),cons(tick(),nil)));
  if (*buf == '"') return parse();
  if (*buf != '(') return quote(parse());
  for (t = cons(atom("list"),nil),p = cell+sp; ; *p = cons(tick(),nil),p = cell+sp) {
    if (scan() == ')') return t;
    if (*buf == '.' && !buf[1]) return scan(),t = cons(atom("append"),cons(t,cons(tick(),nil))),scan(),t;
  }
}

```

16.2 Tail-Call Optimization

Our interpreter implements an evaluation algorithm similar to, but not identical to, McCarthy's classic Lisp evaluator. It uses the functions `eval`, `evlis`, `bind` (a.k.a. `pairlis`), `apply`, `reduce`, and `assoc`. The `eval` function lies at the heart of this. It evaluates a function application represented as a list, say `x`. The list has the function stored at the head `car(x)` and the arguments stored in the rest of the list `cdr(x)`. The steps performed by `eval` and the functions it calls are as follows:

1. `eval(L x,L e)` calls `apply(eval(car(x),e),cdr(x),e)` which evaluates the function at the head with `eval(car(x),e)` first to obtain a closure or primitive to apply to the yet-to-be evaluated arguments stored in the rest of the list `cdr(x)`;
2. `apply(L f,L t,L e)` calls `reduce(f,t,e)` when the function `f` passed to it is a closure, where `t` are the unevaluated arguments to `f` and `e` is the current environment with bindings of symbols to values;
3. `reduce(L f,L t,L e)` calls `bind(car(car(f)),evlis(t,e),not(cdr(f)) ? env : cdr(f))` where the list of the closure's variables `car(car(f))` are pair-wise bound to the list of evaluated arguments `evlis(t,e)` starting with the bindings of the lexical scope of the closure `cdr(f)` if not empty or the global environment `env`, then reduces the closure's body `cdr(car(f))` with a call to `eval` as can be seen in the body of the `reduce` function that returns `eval(cdr(car(f)),bind(car(car(f)),evlis(t,e),not(cdr(f)) ? env : cdr(f)))`.

Note that the sequence of events started with `eval` making certain calls that eventually ended with a call to `eval`. Tail-call optimization aims to remove all intermediate calls that lead to the final `eval` call. Then, rather than calling `eval` again, tail-call optimization loops back the originating `eval`. This means that a Lisp function evaluation at the “tail end” does not incur any return stack growth. For example:

```

(define func
  (lambda (n)
    (func (+ n 1))))

```

is tail-call optimized, because the application `(func2 (+ n 1))` is the body of `func1`. By contrast:

```

(define func
  (lambda (n)
    (+ 1 (func n))))

```

is not tail-call optimized, because the result of `func2` is used by the addition operator to increment the value which is returned by `func1`. Let's break down the following tail-call optimized `eval`:

```

L eval(L x,L e) {
  L f,v,d;
  while (1) {
    if (T(x) == ATOM) return assoc(x,e);
    if (T(x) != CONS) return x;
    f = eval(car(x),e); x = cdr(x);
    if (T(f) == PRIM) return prim[ord(f)].f(x,e);
    if (T(f) != CLOS) return err;
    v = car(car(f)); d = cdr(f);
    if (T(d) == NIL) d = env;
    for (; T(v) == CONS && T(x) == CONS; v = cdr(v),x = cdr(x))
      d = pair(car(v),eval(car(x),e),d);
    for (x = T(x) == CONS ? evlis(x,e) : eval(x,e); T(v) == CONS; v = cdr(v),x = cdr(x))
      d = pair(car(v),car(x),d);
    if (T(v) == ATOM) d = pair(v,x,d);
    x = cdr(car(f)); e = d;
  }
}

```

The code is more convoluted than our original simple and elegant `eval`. It looks like a lot is going on here, but it is not complicated once we dig into it. Important to note is that `eval` loops until `x` is a symbol tagged `ATOM` or it is a constant, meaning something that is not applicable. By contrast, a `CONS` is a function application that we should perform. To do so, `f = eval(car(x),e)` gives us the closure/primitive `f`. We get the rest of the list of arguments with `x = cdr(x)`.

When `f` is a primitive, we call it with `if (T(f) == PRIM) return prim[ord(f)].f(x,e)` to return its value. Note that `ord(f)` is the index of the primitive into the `prim[]` array with function pointer `f` to call member function `f(x,e)` with unevaluated list of arguments `x` and the current environment `e`.

When `f` is a closure, we obtain its list of variables `v = car(car(f))` and lexical scope of bindings `d = cdr(f)`. If this scope is empty, the function's scope is global (see function `reduce` in Section 5) and we set it accordingly `if (T(d) == NIL) d = env`.

The lambda variables bindings to the evaluated arguments are made by two `for`-loops that essentially combine the functionalities of `evlis` and the former `bind`. When `v` is an `ATOM`, instead of `CONS` or `NIL`, we know it appears to the right of the dot operator in the formal argument list. Likewise, when `x` is an `ATOM` it appears to the right of a dot operator in the actual argument list and its value is retrieved with `eval(x,e)` that calls `assoc(x,e)` or returns `x` when it is not an `ATOM`. Finally, we assign the body of the closure `x = cdr(car(f))` to replace `x` to be evaluated next in the updated environment `e = d`.

The optimization got rid of `apply`, `bind` and `reduce` that are no longer needed. This optimized implementation performs the same operations as the previous unoptimized implementation, which can be viewed as a reference implementation that implements the Lisp interpreter requirements¹⁹.

Tail-call optimization is only effective with full garbage collection. Our tiny interpreter does not perform garbage collection continuously, but rather waits until returning to the prompt to reclaim memory. This means that evaluated arguments will continue to accumulate in memory and this will eventually exhaust memory. If we can reclaim the list of evaluated arguments before making

¹⁹We want our Lisp interpreter to support the dot operator in function application lists and in `lambda` variable lists, such as `(lambda (x . args) args)`, but other Lisp may not support these useful forms.

a tail-call, then we will not run out of memory. Some other small Lisp interpreters use a so-called “ABC” garbage collector, which immediately reclaims the evaluated list of arguments passed to a function when the function returns (or tail-calls). It assumes that Lisp data structures are not cyclic. However, this assumption is not a sufficient requirement for “ABC” garbage collection to be sound. Our Lisp interpreter supports forms like `(lambda args args)` and `(lambda (x . args) args)` that use a single variable `args` to refer to the (partial) list of evaluated arguments passed to the closure. The `args` reference is invalidated after “ABC” garbage collection. Hence, “ABC” garbage collection is not safe. The next sections apply more optimizations to reduce memory usage.

When macros are implemented as described previously, then we can get rid of `expand` and add the following code to `eval` just before the `if (T(f) != CLOS) return err(3);` part to check if `f` is a macro that must be expanded first, then evaluated by continuing the evaluation loop in `eval`:

```
if (T(f) == MACR) {
    for (d = env, v = car(f); T(v) == CONS; v = cdr(v), x = cdr(x)) d = pair(car(v), car(x), d);
    if (T(v) == ATOM) d = pair(v, x, d);
    x = eval(cdr(f), d);
    continue;
}
```

The recursive `eval` call runs the macro and the results are then evaluated again (as expanded Lisp code) by continuing the loop.

16.3 Tail-Call Optimization Part Deux

In order to tail-call optimize Lisp evaluation through the special forms `if`, `cond`, and `let*`, we mark these three primitives in the `prim[]` array by introducing a new member `short t` flag with the value 1 for tail-calls:

```
struct { const char *s; L (*f)(L,L*); short t; } prim[] = {
    {"eval",    f_eval,    1},
    {"quote",   f_quote,   0},
    {"cons",    f_cons,    0},
    {"car",     f_car,     0},
    {"cdr",     f_cdr,     0},
    {"+",       f_add,     0},
    {"-",       f_sub,     0},
    {"*",       f_mul,     0},
    {"/",       f_div,     0},
    {"int",     f_int,     0},
    {"<",       f_lt,      0},
    {"eq?",     f_eq,      0},
    {"pair?",   f_pair,    0},
    {"or",      f_or,      0},
    {"and",     f_and,     0},
    {"not",     f_not,     0},
    {"cond",    f_cond,    1},
    {"if",      f_if,      1},
    {"let*",    f_let*,    1},
    {"let",     f_let,     1},
    {"letrec*", f_letrec*, 1},
}
```

```

{"letrec", f_letrec, 1},
{"lambda", f_lambda, 0},
{"define", f_define, 0},
{0}};

```

Perhaps surprisingly, we also make `eval` in `prim[]` a tail-call because it just returns its argument to be evaluated next. When the `t` member flag is set, we continue the `eval` function's loop to evaluate the Lisp expression `x` returned by the tail-call enabled primitive:

```

L eval(L x,L e) {
  L f,v,d;
  while (1) {
    ...
    if (T(f) == PRIM) {
      x = prim[ord(f)].f(x,&e);
      if (prim[ord(f)].t) continue;
      return x;
    }
    ...
  }
}

```

In addition, the `if`, `cond`, and `let*` primitives no longer call `eval` before returning to return the expression to be evaluated instead:

```

L f_cond(L t,L *e) {
  while (not(eval(car(car(t))),*e)) t = cdr(t);
  return car(cdr(car(t)));
}
L f_if(L t,L *e) {
  return car(cdr(not(eval(car(t)),*e) ? cdr(t) : t));
}
L f_let(L t,L *e) {
  L d = *e;
  for (; let(t); t = cdr(t)) *e = pair(car(car(t)),eval(car(cdr(car(t))),d),*e);
  return car(t);
}
L f_letrec(L t,L *e) {
  L *p;
  for (; let(t); t = cdr(t)) p = &cell[ord(car(*e = pair(car(car(t)),nil,*e)))],*p = eval(car(cdr(car(t))),*e);
  return car(t);
}
L f_letrec(L t,L *e) {
  L s,d,*p;
  for (s = t,d = *e,p = &d; let(s); s = cdr(s)) *p = pair(car(car(s)),nil,*e),p = &cell[ord(*p)];
  for (*e = d; let(t); t = cdr(t),d = cdr(d)) cell[ord(car(d))] = eval(car(cdr(car(t))),*e);
  return car(t);
}

```

The `f_let` primitive demonstrates why we pass `*e` to the primitives instead of `e` by value. This permits the primitive to extend the environment to continue evaluation with an expression `x` that has an extended scope of bindings `e`. The rest of the primitives remain the same, except that we pass a pointer to the current environment `*e` and for the updated `f_eval`, as stated earlier:

```

L f_eval(L t,L *e) { return car(evlis(t,*e)); }

```

16.4 Optimizing the Lisp Primitives

The `evlis` function constructs a list of evaluated arguments. It is a fundamental Lisp interpreter function. While optimizing our Lisp interpreter we got rid of several Lisp interpreter functions, but not `evlis`, which is called in `eval` to handle the dot operator in lambda variable lists by binding the list of evaluated arguments to a variable. All other calls to `evlis` in our Lisp interpreter are made to evaluate the arguments passed to a primitive. The important point is that `evlis` correctly handles the dot operator in the arguments passed to a primitive. For example, we can define a `sum` function that calls `+` on its list argument `t`:

```
(define sum (lambda (t) (+ . t)))
(sum '(1 2 3))
6
```

Here, the `evlis` call in `f_add` simply passes the list `t = (1 2 3)` back as a return value to be summed in `f_add`'s loop over its arguments. If we were to blindly remove `evlis` from `f_add` to replace it with a call to `eval` for each unevaluated `car(t)` in the list of unevaluated arguments `t`, then we will lose the ability to pass arguments via the dot operator.

Rather than calling `evlis` in a primitive, we implement an iterative approach that calls a function `evarg` repeatedly. This function returns the next evaluated argument from the list of arguments `t`:

```
L evarg(L *t, L *e, I *a) {
    L x;
    if (T(*t) == ATOM && !*a) *t = assoc(*t,*e), *a = 1;
    x = car(*t); *t = cdr(*t);
    return *a ? x : eval(x,*e);
}
```

We pass a pointer `*t` to `evarg` updated by `evarg` to point to the rest of the list of arguments that are not yet evaluated. The `evarg` function checks if `*t` ends in a symbol, which is the variable after the dot operator. In this case, the value of the variable should be looked up with `assoc` and its list is used as the remaining list of arguments that are already evaluated. When this happens, a flag `*a` is set to prevent double evaluation of dot operator arguments. Now we can use `evarg` to optimize our Lisp primitives, such as `cons`:

```
L f_cons(L t, L *e) { I a = 0; L x = evarg(&t,e,&a); return cons(x,evarg(&t,e,&a)); }
```

A word of caution is in order here. Argument expression evaluation order in C is unspecified, so we cannot `return cons(evarg(&t,e,&a),evarg(&t,e,&a))`, which leads to undefined behavior. We may get the correct `cons` of the two expressions if we're lucky, or we may end up `cons`'ing them swapped. We should be grateful that our Lisp interpreter always evaluates arguments from the left to the right, knowing that this matters if the evaluation of the argument expressions have side-effects, such as printing values to the screen or when using `setq`, `set-car!` and `set-cdr!` that change values, as those operations must be properly ordered.

Using `evarg` suffices to implement all primitives except for variadic primitives that take any number of arguments. In that case we need to repeatedly check if a next argument is present to evaluate. Let's define `isarg` to do just that:

```

I isarg(L *t,L *e,I *a,L *x) {
  if (T(*t) == ATOM && !*a) *t = assoc(*t,*e),*a = 1;
  if (T(*t) != CONS) return 0;
  *x = car(*t); *t = cdr(*t);
  if (!*a) *x = eval(*x,*e);
  return 1;
}

```

The only difference with `evarg` is that `isarg` returns nonzero if an argument is present and assigns the evaluated value to `*x` returned to the caller via a pointer. Otherwise it returns zero and leaves `*x` unassigned. We could rewrite `evarg` to use `isarg` and return an appropriate error when an argument is missing. Using `evarg` and `isarg` the Lisp primitives are modified as follows:

```

L f_eval(L t,L *e) { I a = 0; return evarg(&t,e,&a); }
L f_quote(L t,L *_ ) { return car(t); }
L f_cons(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return cons(x,evarg(&t,e,&a)); }
L f_car(L t,L *e) { I a = 0; return car(evarg(&t,e,&a)); }
L f_cdr(L t,L *e) { I a = 0; return cdr(evarg(&t,e,&a)); }
L f_add(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n += x; return num(n); }
L f_sub(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n -= x; return num(n); }
L f_mul(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n *= x; return num(n); }
L f_div(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n /= x; return num(n); }
L f_int(L t,L *e) { I a = 0; L n = evarg(&t,e,&a); return n < 1e16 && n > -1e16 ? (long long)n : n; }
L f_lt(L t,L *e) { I a = 0; L n = evarg(&t,e,&a); return n - evarg(&t,e,&a) < 0 ? tru : nil; }
L f_eq(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return equ(x,evarg(&t,e,&a)) ? tru : nil; }
L f_pair(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return T(x) == CONS ? tru : nil; }
L f_or(L t,L *e) { I a = 0; L x = nil; while (isarg(&t,e,&a,&x) && not(x)) continue; return x; }
L f_and(L t,L *e) { I a = 0; L x = tru; while (isarg(&t,e,&a,&x) && !not(x)) continue; return x; }
L f_not(L t,L *e) { I a = 0; return not(evarg(&t,e,&a)) ? tru : nil; }

```

Another benefit of this optimization is that `or` and `and` now respect the dot operator, just like the arithmetic functions. Special forms such as `cond` and `if` should not be changed to respect the dot operator, so we leave them alone.

We also update `eval` to use `evarg` to evaluate the actual arguments that are passed to a closure, which involves extending the environment `e` with local bindings `d`. Because `evarg` respects the dot operator, this part of the code is simplified also:

```

L eval(L x,L e) {
  I a; L f,v,d;
  while (1) {
    if (T(x) == ATOM) return assoc(x,e);
    if (T(x) != CONS) return x;
    f = eval(car(x),e); x = cdr(x);
    if (T(f) == PRIM) {
      x = prim[ord(f)].f(x,&e);
      if (prim[ord(f)].t) continue;
      return x;
    }
    if (T(f) == MACR) {
      for (d = env,v = car(f); T(v) == CONS; v = cdr(v),x = cdr(x)) d = pair(car(v),car(x),d);
      if (T(v) == ATOM) d = pair(v,x,d);
      x = eval(cdr(f),d);
    }
  }
}

```

```

        continue;
    }
    if (T(f) != CLOS) return err(3);
    v = car(car(f)); d = cdr(f);
    if (T(d) == NIL) d = env;
    for (a = 0; T(v) == CONS; v = cdr(v)) d = pair(car(v), evarg(&x, &e, &a), d);
    if (T(v) == ATOM) d = pair(v, a ? x : evlis(x, e), d);
    x = cdr(car(f)); e = d;
}
}

```

These optimizations increase the speed of Lisp evaluation and reduces memory usage. However, the code is more convoluted and harder to understand compared to the original.

Previously we added execution tracing to our interpreter with a new **trace** primitive and a new internal function **step** that replaces **eval**. For the optimized **eval** version however, we will end up not tracing the intermediate steps as we go through the loop. Instead, let's add a new internal function **trace(y,x,e)** that shows the evaluation step from expression **y** to the evaluated value **x**, where **y** is the previous **x** to evaluate and **x** is the newly evaluated value in the **eval** function loop. Then we can add calls to **trace** when **tr** is nonzero. A few other minor changes are needed to replace **return x** with **break** to trace the result **x** by breaking out of the loop:

```

I tr = 0; /* tracing off (0), on (1), wait (2), dump and wait (3) */
...
L f_trace(L t, L *_){ tr = not(t) ? !tr : (I)num(car(t)); return num(tr); }
... prim[] = { ... {"trace", f_trace, 0}, ... };
...
L eval(L x, L e) {
    I a, s = sp; L f, v, d, y;
    while (1) {
        y = x;
        if (T(x) == ATOM) { x = assoc(x, e); break; }
        if (T(x) != CONS) break;
        f = eval(car(x), e); x = cdr(x);
        if (T(f) == PRIM) {
            x = prim[ord(f)].f(x, &e);
            if (prim[ord(f)].t) continue;
            break;
        }
    }
    ...
    if (tr) trace(s, y, x, e);
}
if (tr && !equ(x, y)) trace(s, y, x, e);
return x;
}

```

The **trace** function takes an additional argument **s** that points to the stack location we start with when **eval** is called. We use this location to only dump the memory region between **s** and **sp** that is actually used during the evaluation. Argument **e** is the current local environment displayed by the memory **dump**. To make the tracing and memory dumping stand out from the usual input and output expressions displayed by the REPL on the terminal screen, we use ANSI escape codes²⁰ to colorize the trace and the memory dump:

²⁰https://en.wikipedia.org/wiki/ANSI_escape_code

```

void trace(I s,L y,L x,L e) {
  if (tr > 2) dump(sp,s,e);
  printf("\n\e[32m%u \e[33m",sp); print(y);
  printf("\e[36m => \e[33m"); print(x); printf("\e[m\t");
  if (tr > 1) while (getchar() >= ' ') continue;
}

```

The `dump` function is called at level 3 tracing to dump memory from stack location `i` up to `k`:

```

void dump(I i,I k,L e) {
  if (i < k) {
    printf("\n\e[35m==== DUMP ====");
    while (i < k--) {
      printf("\n\e[35m%s\e[32m%u \e[35m",k-1 == ord(e) ? "local env:\n" :
        k-1 == ord(env) ? "env:\n" : "",k);

      switch (T(cell[k])) {
        case ATOM: printf("ATOM "); printf("\e[32m%u ",ord(cell[k])); break;
        case PRIM: printf("PRIM "); break;
        case CONS: printf("CONS "); printf("\e[32m%u ",ord(cell[k])); break;
        case CLOS: printf("CLOS "); printf("\e[32m%u ",ord(cell[k])); break;
        case MACR: printf("MACR "); printf("\e[32m%u ",ord(cell[k])); break;
        case NIL:  printf("NIL "); break;
        default:   printf(" "); break;
      }
      printf("\e[33m"); print(cell[k]); printf("\e[m%s",k % 2 ? "" : "\n");
    }
    printf("\e[35m===== \e[m\t");
  }
}

```

The colors are viewable in dark and light-themed terminal screens, but might be less legible on bright backgrounds. A few changes to the escape codes should improve that.

16.5 Tail-Calls à Trois

In the previous sections we optimized `eval` in our Lisp interpreter by tail-call optimizing `eval` as a continuous evaluation loop. We also observed that some Lisp special forms, such as `cond`, `if` and `let*` fit the tail-call pattern. By tail-call optimizing them we pass their unevaluated return expression back into the evaluation loop. Because of these optimizations, `cond`, `if`, `let*` and the additional `let` and `letrec` special forms are efficiently executed without unnecessary stack operations, even when nested together in code in specific ways.

In fact, lambda closures are also tail-call optimized in the continuous evaluation loop. However, passing arguments to a closure builds up a list of variable bindings on the stack, which serves as the local environment to evaluate the body of the closure. Therefore, the stack will grow, even when closures are recursive. In this third part we will also optimize tail-recursive lambda closures, i.e. closures that recursively call themselves.

Tail-recursion happens when a lambda closure calls itself without any operations performed by the caller on the callee's return value. Consider for example the definition of `begin`:

```
(define begin (lambda (x . args) (if args (begin . args) x)))
```

Clearly, **begin** is tail-recursive. The **begin** function (also called **progn** in some other Lisp) evaluates all arguments from the left to the right, but only returns the value of the last²¹. So it sequences several Lisp expressions (typically with side-effects) as statements. For example:

```
(begin (println 1) (println 2))
```

Even though **begin** is tail-recursive, the stack still grows with each recursive call. This happens because **eval** builds up a list of local bindings **d** on the stack by iterating over the list of lambda variables **v**. In the case of **begin**, its **x** is bound to the first argument and **args** is bound to the remaining list of arguments. Each new binding takes up four cells on the stack, by performing two **cons** calls for each **pair** call in **eval**. Therefore, each recursive **begin** adds eight cells to the stack. Tail-recursion optimization avoids this growth by reusing the stack space of previous bindings that are no longer needed. To achieve this, we increase **sp** by 4 for each variable binding that we pop from the stack to reassign its value to the variable's previous binding located deeper on the stack. We must be careful to only pop binding pairs of environment **d** consecutively from the stack and nothing else. Otherwise, we will destroy list data structures on the stack that are associated with (other) variables that may still be in use.

To detect tail-recursive closures in **eval**, we save closure **f** to a new variable **g** in **eval** before we loop again. At this point we also save the current environment **e** to a new variable **h**. This allows us to keep track of all new bindings made in a recursive call, including additional **let**-local bindings, when applicable. A tail-recursive function call is made when **equ(f,g)**. In this case, the local environment **d** constructed for closure **f** has parent environment **e** of the previous **g** containing its variables and **let**-locals:

```
if (equ(f,g)) d = e;
```

Then we bind closure **f** variables **v** to the evaluated argument values **x** exactly like before in **eval**:

```
for (a = 0; T(v) == CONS; v = cdr(v)) d = pair(car(v), evarg(&x, &e, &a), d);
if (T(v) == ATOM) d = pair(v, a ? x : evlis(x, e), d);
```

For tail-recursive closure **f**, we first reassign the closure's previous variable bindings in environment **e** to their new values stored in environment **d** and delete the obsolete bindings of **d**. Second, we delete the **let**-locals of closure **g**. Deleting is safe as long as **sp == ord(d)** to pop 4 cells of a binding in **d** and update **d = cdr(d)**. Ensuring consecutive pops when walking **d** avoids destroying other list data structures on the stack that may be located between bindings:

```
if (equ(f,g)) {
    for (; !equ(d,e) && sp == ord(d); d = cdr(d), sp += 4) assign(car(car(d)), cdr(car(d)), e);
    for (; !equ(d,h) && sp == ord(d); d = cdr(d)) sp += 4;
}
```

The **assign** function is similar to **setq**, but doesn't evaluate any arguments. It is solely used to move the new value **x** of a variable **v** to the previous binding of **v** in environment **e**:

```
void assign(L v, L x, L e) {
    while (!equ(v, car(car(e)))) e = cdr(e);
    cell[ord(car(e))] = x;
}
```

²¹(**begin** *expr*) $\equiv$ *expr*, thus **begin** also serves as the *identity function* in its (tail-call optimized) last argument!

And... that's it! Now **begin** and other tail-recursive lambda closures no longer grow the stack unnecessarily. Combining this all together in **eval** we have:

```
L eval(L x,L e) {
  I a,s = sp; L f,v,d,y,g = nil,h;
  while (1) {
    y = x;
    if (T(x) == ATOM) { x = assoc(x,e); break; }
    if (T(x) != CONS) break;
    f = eval(car(x),e); x = cdr(x);
    if (T(f) == PRIM) {
      x = prim[ord(f)].f(x,&e);
      if (prim[ord(f)].t) continue;
      break;
    }
    if (T(f) == MACR) {
      for (d = env,v = car(f); T(v) == CONS; v = cdr(v),x = cdr(x)) d = pair(car(v),car(x),d);
      if (T(v) == ATOM) d = pair(v,x,d);
      x = eval(cdr(f),d);
      continue;
    }
    if (T(f) != CLOS) return err(3);
    v = car(car(f));
    if (equ(f,g)) d = e;
    else if (not(d = cdr(f))) d = env;
    for (a = 0; T(v) == CONS; v = cdr(v)) d = pair(car(v),evarg(&x,&e,&a),d);
    if (T(v) == ATOM) d = pair(v,a ? x : evlis(x,e),d);
    if (equ(f,g)) {
      for (; !equ(d,e) && sp == ord(d); d = cdr(d),sp += 4) assign(car(car(d)),cdr(car(d)),e);
      for (; !equ(d,h) && sp == ord(d); d = cdr(d)) sp += 4;
    }
    x = cdr(car(f)); e = d; g = f; h = e;
    if (tr) trace(s,y,x,e);
  }
  if (tr && !equ(x,y)) trace(s,y,x,e);
  return x;
}
```

With this machinery in place, tail-recursive Lisp functions with **let**-local variables don't run out of stack space. For example, we can define a tail-recursive factorial function **f** in a **letrec*** with **let**-locals to illustrate and wrap it in a **fact** definition as follows:

```
(define fact
  (lambda (n)
    (letrec*
      (f (lambda (r k)
          (let
            (k1 (- k 1))
            (rk (* r k))
            (if (< k 2)
              r
              (f rk k1))))))
      (f 1 n))))
```

In case you want the best performance to sequence statements with `begin`, you could add a new primitive `progn`, a name that has historical roots in Lisp, that is an efficient tail-call optimized built-in special-form of `begin`:

```
L f_progn(L t,L *e) {
    for (; let(t); t = cdr(t)) eval(car(t),*e);
    return car(t);
}
.. prim[] = { ... {"progn",f_progn,1} ... };
```

16.6 Higher-Order Functions, Tail-Recursive Functions and Loops

Recursion is a powerful concept. Tail-recursive functions are as efficient as loops. Moreover, recursive *higher-order functions* that apply a given function to lists, such as `foldl`, `foldr`, and `map` (see Appendix E.3), are building blocks for more complex functions. For example, `map` returns a new list in which a function is applied to each value of a a given list or lists. Using `map` we can print 1 to 9 by mapping `println` to each value in the `(seq 1 10)` list:

```
(map println (seq 1 10))
```

Mapping with lambdas lets you map any function without having to define the function globally. For example, to construct a list of squared values, we can `map` a lambda:

```
(map (lambda (k) (* k k)) (seq 1 10))
```

Now, let's define `make-list` that creates a list of `n` copies of a given value `x`:

```
(define make-list
  (lambda (n x)
    (map (lambda () x) (seq 0 n))))
```

For example, `(make-list 3 'sheep)` gives `(sheep sheep sheep)`. Without `map` we can define a recursive `make-list` function, but it looks clunky compared to the `map` version:

```
(define make-list
  (lambda (n x)
    (if (< 0 n)
        (cons x (make-list (- n 1) x))
        ())))
```

This `make-list` version is not tail recursive as it `cons`'es the list after each recursive call. To make it tail-recursive, we place the `cons`'ing of the list into the recursive call as an argument²²:

```
(define make-list-tr
  (lambda (n x t)
    (if (< 0 n)
        (make-list-tr (- n 1) x (cons x t))
        t)))
(define make-list (lambda (n . x) (make-list-tr n (if x (car x) ()) ())))
```

²²This constructs the list in reverse, but we don't care since all `make-list` values in the list are the same.

Moving the operations performed after each recursive call to one (or more) extra arguments in the recursive call is a powerful technique. This tail-recursive definition is efficient, but lacks clarity compared to the `map` version. Or perhaps we can do better if we have loops to our disposal in our Lisp? But our Lisp has no loop primitives you say! Well, why not, let's add one:

```
L f_while(L t, L *e) {
  L s,x = nil;
  while (!not(eval(car(t),*e)))
    for (s = cdr(t); T(s) == CONS; s = cdr(s)) x = eval(car(s),*e);
  return x;
}
.. prim[] = { ... {"while",f_while,0} ... };
```

For example, we can use a `while`-loop to print the values 1 to 9 squared, using a `let*` with a local loop counter `k` that is updated with a destructive `setq`:

```
(let* (k 1)
  (while (< k 10)
    (println (* k k))
    (setq k (+ k 1))))
```

Note that a `while` loop is a Lisp expression too and returns the last value computed, which is a final value 10 returned by `setq`. With a `while`-loop and destructive `setq` we can implement an iterative version of `make-list`. The second `setq` in the `while`-loop collects the `make-list` list to return at the final iteration:

```
(define make-list
  (lambda (n x)
    (let* (t ())
      (while (< 0 n)
        (setq n (- n 1))
        (setq t (cons x t))))))
```

It's not the best-looking version of `make-list` for sure, but it gets the job done. When programming in Lisp we rarely feel tempted to regress into (ab)using loops with side-effects, instead of using proper (tail) recursion and higher-order functions.

However, if looping with an iterator is something we want to do more often, then we could define a convenient macro (`for k a b expr1 ... exprn`) to construct a `while` loop that iterates a given loop counter name `k` from a value `a` to a value `b` over the expressions `expr`:

```
(define for
  (macro (k a b . args)
    `(let (,k (- ,a 1)) (_ (+ ,b 1))
      (while (< (setq ,k (+ ,k 1)) _) . ,args))))
```

We use backquoting in this macro to construct the `let-while`-structured code instantiated with the given macro arguments `,k`, `,a`, `,b` and `,args` substituted-in using the comma operator. The dotted macro arguments `. args` take multiple expressions given as macro arguments. Note that the `let` in the `for` macro has a local variable strangely named `_` that is assigned the final loop

counter value `b` plus one. This serves an important purpose. We don't want expression `b` to be evaluated in every loop iteration, which would happen if we had used `b` directly in the `while` loop's comparison. When defining macros we should be careful to use and place macro arguments to maintain their evaluation order (especially in the presence of side-effects) and avoid unnecessary reevaluations. This necessitates the use of a `let*` or `let` in the macro, as in this case.

With our `for` macro the loop-based `make-list` definition becomes reasonably understandable. It starts with an empty list in local variable `t`. Iteratively updated with `(setq t (cons x t))`:

```
(define make-list
  (lambda (n x)
    (let* (t ())
      (for i 1 n (setq t (cons x t))))))
```

The reason we picked the name `_` for a local `let` variable in our `for` macro is to obfuscate it from the macro arguments. Things can go horribly wrong when local names clash with names in macro argument expressions that are literally substituted into the macro body. For example, if we replace the loop counter `i` with `_` in `(for _ 1 n (setq t (cons x t)))` then we run into disaster! We can see this from the actual representation of `make-list` after expanding the `for` macro:

```
(define make-list
  (lambda (n x)
    (let* (t ())
      (let (_ (- 1 1)) (_ (+ n 1))
        (while (< (setq _ (+ _ 1)) _) (setq t (cons x t)))))))
```

Obfuscation is ugly and can't fully protect us from naming conflicts. By their nature, the problem is that macro definitions are not as clean as function definitions with respect to variable naming. We want *hygienic macros*. We will add automatic hygienic macros to our Lisp in the next section.

17 Mad Magical Macro Metaprogramming

In the previous section we saw how the `for` macro “extends” Lisp syntax with an iteration-controlled loop structure. Macros are useful to simplify programming when their use cases are compelling and intuitive. Macros can even define new macros in Lisp:

```
(define defmacro
  (macro (f v x)
    `(define ,f (macro ,v ,x))))
```

where the backquoted `(define ,f (macro ,v ,x))` is the same as `(list (quote define) f (list (quote macro) v x))`. The macro application `(defmacro f v x)` with *f* the macro name, *v* the list of macro argument variables, and *x* the macro body expression, expands into the definition `(define f (macro v x))`. With `defmacro` we can define a `defun` function-defining macro:

```
(defmacro defun (f v x)
  `(define ,f (lambda ,v ,x)))
```

The macro application `(defun f v x)` expands into the definition `(define f (lambda v x))`.

These are some simple examples of “template-driven macros” that we may encounter in programming languages that support macro facilities by textual substitution. But Lisp macros are much more powerful than that. We can use them to construct Lisp code in Lisp. That’s what we mean when we say that Lisp is a *homoiconic* programming language. It can manipulate its own code as data, enabling powerful metaprogramming capabilities. This includes code-writing Lisp macros, introspection of Lisp code as data in Lisp, Lisp interpreters and compilers written in Lisp are some examples of this powerful capability. For example, let’s define a **case** construct in Lisp as a macro (**case** *x* (*y*₁ *z*₁) (*y*₂ *z*₂) ... (*y*_{*k*} *z*_{*k*})) to represent the following **let*-cond** code:

```
(let* (_ x)
  (cond
    ((eq? _ y1) z1)
    ((eq? _ y2) z2)
    :
    ((eq? _ yk) zk)
    (#t ())))
```

So that for example (**case** 2 (1 'one) (2 'two) (3 'three)) returns **two**. Note that we use an obfuscated local variable **_** to prevent *capturing* exposed *free variables* in expressions *y_i* and *z_i*. *Hygienic macros* (also mentioned in the previous section) don’t have this problem. We will implement hygienic macros later in our Lisp interpreter.

The **case** macro defined below uses a local **letrec***-recursive code-constructing function **cc** that recurses over the list **t** of (*y_i* *z_i*) pairs of the **case**. That is, *y_i* = ,(car (car t)) and *z_i* = ,(car (cdr (car t))), where the comma operator evaluates these two expressions in the backquoted `(((eq? _ ,(car (car t))) ,(car (cdr (car t)))) . ,(cc (cdr t))) and the dot operator appends the recursively-constructed rest of the list ,(cc (cdr t)) in the **case** macro:

```
(defmacro case (x . args)
  (letrec*
    (cc (lambda (t)
         (if t
             `(((eq? _ ,(car (car t))) ,(car (cdr (car t)))) . ,(cc (cdr t)))
             `(#t ())))))
    `(let* (_ ,x) (cond . ,(cc args)))))
```

Recursion bottoms-out at `(#t ()) appended at the end of the resulting **let\*-cond** code, constructed and returned with `(let* (_ ,x) (cond . ,(cc args)))

Don’t worry if you didn’t get all this in one reading. The point is that this incredible Lisp metaprogramming capability is made possible in our Lisp **eval** interpreter that applies macros with only two lines of C code in Section 11.5 and four lines of C code in the tail-call optimized **eval** in Section 16.2. These few lines of C code make macros “magical”. When a macro is applied, the interpreter assigns the argument *expressions* to macro argument variables, then evaluates the macro body with the local variable bindings that resolve into *expressions*. This mechanism is called *pass by name*. By contrast, Lisp function applications *pass by value* the evaluated arguments as *values* to the function body.

All of these steps are performed at runtime by **eval** in our Lisp interpreter. This isn’t very efficient. As our Lisp programs grow in size with more global definitions, the run-time overhead to look up a name with **assoc** in the global environment **env** grows. *Late binding* global names

gets in the way of performance. Generally speaking, *early binding* that resolves names before a program is run is associated with greater execution speed. By contrast, *late binding* is associated with greater programming flexibility. Promoting one over the other is a trade-off between flexibility versus execution speed of a programming language.

In the remainder of this section we will build a macro-expanding pre-processor `expand` that expands macros at “compile time”, i.e. when code and function definitions are parsed from source input, to completely eliminate the overhead of runtime macro expansion. Macros normally require local name obfuscation, which is madness when we must obfuscate many names. To sanitize macros from dirty obfuscation, we will implement *hygienic macros*.

17.1 Early Binding and Efficient Macro Expansion

The *binding time* of a name depends on scope. *Late binding* is necessary when the name is used as a local variable, since the name will be bound to a value computed at runtime. *Early binding* is only possible for global names whose corresponding cell(s) never change at runtime. That is, it is safe to assume that `setq` is never used at runtime to reassign global functions, primitives and macros. After all, doing so will likely break a running program²³.

Therefore, we will early bind global names to code, but we still late bind global names to data. The type of cells that are associated with Lisp code are `PRIM` cells (built-in primitives), `CLOS` cells (functions), `MACR` cells (macros). Global definitions with these cell values are assumed to remain unchanged during the execution of a program, i.e. runtime reassignment by `setq` is not permitted.

Our objective is to implement a macro-expanding pre-processor `expand(L x, L e, L b)` to transform a Lisp code expression `x` by expanding macro applications in `x` and to optimize function applications in `x` by early binding the names of Lisp built-ins (`PRIM` primitives) and user-defined global functions (`CLOS` closures). We use `expand` in the REPL as an extra step, that is, a Read-Expand-Eval-Print Loop (a “REEPL”):

```
while (1) { ... print(eval(expand(Read()),env,nil),env)); }
```

Our new `expand` function in C has the following outline:

```
L expand(L x, L e, L b) {
    L c,d,v,w,y;
    if (T(x) == ATOM) { ... }          /* case 1: the name of a variable */
    if (T(x) == CLOS) { ... }          /* case 2: a function (closure) */
    if (T(x) == CONS) {
        L f = expand(car(x),e,b);
        L t = cons(f,nil),*p = cell+sp;
        x = cdr(x);
        if (T(f) == MACR) { ... }       /* case 3: the application of a macro */
        if (T(f) == PRIM) { ... }       /* case 4: the application of a special-form primitive */
        for (; T(x) == CONS; x = cdr(x),p = cell+sp) *p = cons(expand(car(x),e,b),nil);
        *p = expand(x,e,b);
        return t;
    }
    return x;
}
```

²³A Lisp program should use lambdas to selectively apply functions and to construct new functions, even global ones, e.g. `(define afun nil) then (setq afun (lambda ...))` works around this restriction.

Argument `x` of `expand` is the Lisp code expression to expand, `e` is the environment that represents the lexical scope of `x`, and `b` is the environment with macro argument-expression bindings to facilitate macro expansion. The `expand` function recurses over the function `f` and its arguments in `x` when `x` is a list. If we ignore cases 1 to 4 for now, which we will be built shortly, then we see that the final loop in `expand` constructs a new function application of the expanded function `f` (primitive or closure) applied to the expanded arguments in the list `x`. This is the base case for Lisp function applications represented by lists using the `CONS` cell. The transformed new list `t` is constructed using pointer `*p` to `cons` at the back of `t`, a technique we've used before.

The recursive `expand` function analyzes Lisp code and transforms (“compiles”) the code by early binding globals when in scope and by expanding macros. The scope of variables applicable to expression `x` is important. A `let` with a local name may *shadow* the name of a global, i.e. hiding the global name by the same name used in a local lexical scope. Therefore, we must keep track of locals declared in a `lambda` as arguments and the locals declared in `let*`, `let`, `letrec*`, and `letrec`. To do so, we use *abstract interpretation* to interpret the code to transform without actually running the code. We use this powerful form of interpretation to analyze the lexical scope, by storing lexical scope bindings in environment `e`. Initially `e` is the global environment `env` and the environment `b` of macro argument bindings is empty `nil`.

To look up a name in environment `e` or in `b` in `expand`, we define `lookup` to return nonzero when the name of a variable `v` is bound in the given environment to `e` to search. When found, pointer `*x` is set to this value to return it. Otherwise, if the name is not bound, then `lookup` returns zero:

```
I lookup(L v,L e,L *x) {
  while (T(e) == CONS && !equ(v,car(car(e)))) e = cdr(e);
  if (T(e) != CONS) return 0;
  *x = cdr(car(e));
  return 1;
}
```

The first case in `expand` is to analyze and transform the name of a variable, i.e. expression `x` is an `ATOM` cell:

```
if (T(x) == ATOM) {
  /* case 1: the name of a variable */
  if (lookup(x,b,&y)) return y;
  if (lookup(x,e,&y) && (T(y) == PRIM || T(y) == CLOS || T(y) == MACR)) return y;
  return x;
}
```

This looks up name `x` in the environment of macro argument bindings `b` first. When found, the transformed code `y` is the expression `y` bound to macro argument name `x`. Otherwise, this looks up name `x` in the lexical scope environment `e`. We are only interested in global definitions of name `x` bound to a primitive, or to a function, or to a macro, which when found is returned as `y`. This suffices to early bind globals that are primitives, functions, and macros. For example, when we enter the Lisp expression (`not #t`) at the REPL prompt, then `expand` transforms this into (`<not> #t`), since `not` is a built-in primitive and `#t` is bound to itself as an `ATOM` cell.

For the next case, when `x` is a closure, `expand` analyzes the closure's list of argument variables `v` to construct environment `d` with the lexical scope of bindings and also to construct updated environment `c` with bindings `b` of macro argument variables that may be shadowed by variables in `v`. A new closure(`w,expand(y,d,c),e`) with the closure body `y = cdr(car(x))` expanded is returned:

```

if (T(x) == CLOS) {
    /* case 2: a function (closure) */
    v = w = car(car(x));
    d = e = cdr(x);
    y = cdr(car(x));
    if (T(d) == NIL) d = env;
    for (c = b; T(v) == CONS; v = cdr(v)) d = pair(car(v),nil,d),c = pair(car(v),car(v),c);
    if (T(v) == ATOM) d = pair(v,nil,d),c = pair(v,v,c);
    return closure(w,expand(y,d,c),e);
}

```

Note that environment `d` has each closure argument variable `car(v)` bound to `nil` to hide shadowed globals. Environment `c` has each closure argument variable `car(v)` bound to itself, which expands to its original name as an effective mechanism to hide shadowed macro arguments.

To expand the application of primitives, functions and macros represented by the `CONS` cell `x`, `expand` first transforms the function part `f = expand(car(x),e,b)` to construct the transformed, but incomplete, application list `t = cons(f,nil)`. List `t` is constructed by `cons`'ing at the back of `t` with pointer `*p`.

When `f` is a macro to apply, we construct a new environment of macro argument bindings `c` by iterating over the macro argument variables `v`. Expanding the macro using macro argument binding environments `c` and `b` takes three steps. First by substituting the macro arguments by their expressions in the macro body `cdr(f)` with `expand(cdr(f),e,c)`. Then by evaluating the result with `eval`. After all, a macro must create Lisp code by running the Lisp macro body. The resulting Lisp code is expanded a final time with `expand(...,e,b)` to be returned:

```

if (T(x) == CONS) {
    L f = expand(car(x),e,b);
    L t = cons(f,nil),*p = cell+sp;
    x = cdr(x);
    if (T(f) == MACR) {
        /* case 3: the application of a macro */
        for (c = nil,v = car(f); T(v) == CONS; v = cdr(v),x = cdr(x)) c = pair(car(v),quote(car(x)),c);
        if (T(v) == ATOM) c = pair(v,quote(x),c);
        return expand(eval(expand(cdr(f),e,c),e),e,b);
    }
}

```

Note that `quote(x)` (defined in Section 16.1) quotes `x` by constructing the expression `(quote x)`. This is necessary because `eval` should not evaluate the expressions substituted into the macro body as macro arguments by the inner first call to `expand(cdr(f),e,c)` with environment `c`.

When `f` is a special-form primitive to apply, abstract interpretation is used to interpret the code to analyze and establish the lexical scope of `lambda` argument variables and `let*`, `let`, `letrec*`, and `letrec` necessary to expand and transform their body expressions. We should not expand a `quote` application, but we can expand the body expression of a `macro` application:

```

if (T(x) == CONS) {
    L f = expand(car(x),e,b);
    L t = cons(f,nil),*p = cell+sp;
    x = cdr(x);
    if (T(f) == MACR) { ... } /* case 3: the application of a macro */
    if (T(f) == PRIM) {
        /* case 4: the application of a special-form primitive */
        if (equ(f,p_quote)) { *p = x; return t; }
        if (equ(f,p_macro)) { *p = cons(car(x),cons(expand(car(cdr(x)),e,b),nil)); return t; }
        if (equ(f,p_lambda)) { ... }
    }
}

```

```

    if (equ(f,p_cond)) { ... }
    if (equ(f,p_let)) { ... }
    if (equ(f,p_letrec)) { ... }
    if (equ(f,p_letrec)) { ... }
}
for (; T(x) == CONS; x = cdr(x),p = cell+sp) *p = cons(expand(car(x),e,b),nil);
*p = expand(x,e,b);
return t;
}
return x;
}

```

To identify these primitives, such as `equ(f,p_quote)`, we need the following C code additions to the Lisp interpreter:

```
L p_quote,p_lambda,p_macro,p_cond,p_let,p_letrec,p_letrec;
```

We assign these PRIM cell variables right after the populating `env` in the `main` function as follows:

```

for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
p_quote  = assoc(atom("quote"),env);
p_lambda = assoc(atom("lambda"),env);
p_macro  = assoc(atom("macro"),env);
p_cond   = assoc(atom("cond"),env);
p_let    = assoc(atom("let"),env);
p_letrec = assoc(atom("letrec"),env);
p_letrec = assoc(atom("letrec"),env);

```

The `expand` code continues to analyze and transform special forms. The `lambda` special form has a list of argument variables `v = car(x)` and a body expression `car(cdr(x))`. Augmented environments `d` and `c` of `e` and `b` are constructed, respectively, to obey lexical scoping of `lambda` arguments in the `lambda` body. This effectively hides shadowed globals in `e` and hides shadow macro arguments in `b` by using their augmented environments `d` and `c`, respectively:

```

if (equ(f,p_lambda)) {
    v = car(x);
    *p = cons(v,nil);
    for (d = e,c = b; T(v) == CONS; v = cdr(v)) d = pair(car(v),nil,d),c = pair(car(v),car(v),c);
    if (T(v) == ATOM) d = pair(v,nil,d),c = pair(v,v,c);
    cell[ord(*p)] = cons(expand(car(cdr(x)),d,c),nil);
    return t;
}

```

The `cond` special form is straightforward. We just need to make sure that the `cond` condition-expression pairs are not transformed as function applications, but are retained as pairs of two transformed expressions:

```

if (equ(f,p_cond)) {
    for (; T(x) == CONS; x = cdr(x),p = cell+sp)
        *p = cons(cons(expand(car(car(x))),e,b),cons(expand(car(cdr(car(x)))),e,b),nil)),nil);
    return t;
}

```

The `let*` special form has a local lexical scope of variables followed by the locally-scoped `let*` body expression. The lexical scope is represented by environments `d` and `c` that augment `d` and `b`, respectively with bindings to each variable `v = car(car(x))`. The `let*` expressions `car(cdr(car(x)))` are transformed with `expand` using environments `d` and `c` that capture the local scope `let*` semantics that serially binds variables to their values:

```
if (equ(f,p_leta)) {
  for (d = e,c = b; let(x); x = cdr(x),p = &cell[ord(*p)]) {
    v = car(car(x));
    *p = cons(cons(v,cons(expand(car(cdr(car(x))),d,c),nil)),nil);
    d = pair(v,nil,d); c = pair(v,v,c);
  }
  *p = cons(expand(car(x),d,c),nil);
  return t;
}
```

The other `let`, `letrec*` and `letrec` special forms are transformed similarly:

```
if (equ(f,p_let)) {
  for (d = e,c = b; let(x); x = cdr(x),p = &cell[ord(*p)]) {
    v = car(car(x));
    *p = cons(cons(v,cons(expand(car(cdr(car(x))),e,c),nil)),nil);
    d = pair(v,nil,d); c = pair(v,v,c);
  }
  *p = cons(expand(car(x),d,c),nil);
  return t;
}
if (equ(f,p_letreca)) {
  for (d = e,c = b; let(x); x = cdr(x),p = &cell[ord(*p)]) {
    v = car(car(x));
    d = pair(v,nil,d); c = pair(v,v,c);
    *p = cons(cons(v,cons(expand(car(cdr(car(x))),d,c),nil)),nil);
  }
  *p = cons(expand(car(x),d,c),nil);
  return t;
}
if (equ(f,p_letrec)) {
  for (d = e,c = b,y = x; let(y); y = cdr(y)) {
    v = car(car(y));
    d = pair(v,nil,d); c = pair(v,v,c);
  }
  for (; let(x); x = cdr(x),p = &cell[ord(*p)]) {
    v = car(car(x));
    *p = cons(cons(v,cons(expand(car(cdr(car(x))),d,c),nil)),nil);
  }
  *p = cons(expand(car(x),d,c),nil);
  return t;
}
```

Adding the macro-expanding pre-processor significantly speeds up Lisp programs by eliminating unnecessary runtime calls to `assoc` for globals, which is accomplished by early binding globals beforehand. Also macro expansion overhead is eliminated. For example, the `case` macro that we defined earlier, used in a `digit` function:

```
(defun digit (d)
  (case d
    (1 'one)
    (2 'two)
    (3 'three)))
```

transforms to an optimized lambda closure by constructing the following `let*-cond` Lisp code at “compile time”:

```
(<lambda> (d)
  (<let*> (n d)
    (<cond>
      ((<eq?> n 1) (<quote> one))
      ((<eq?> n 2) (<quote> two))
      ((<eq?> n 3) (<quote> three))
      (#t ())))))
```

What we’ve accomplished with our new additions to our Lisp interpreter is an efficient macro expansion mechanism with “compile-time” early bindings for built-in primitives and globally-defined functions. The names of globally-defined functions are replaced with their closures, provided that functions are defined before used so that their definitions are visible at “compile time”.

By contrast, when a globally-defined function is not defined before used, then function invocation requires an `assoc` call in `eval` to look up its name at runtime, which is still slow. Unfortunately, this late-binding `assoc` runtime overhead also happens for recursive functions, because the function is not yet defined in the global environment when `expand` visits the recursive function body and therefore can’t replace the name with its closure in the function body. We can remedy this situation by adding a dummy closure paired with the name of the function to the environment, `expand` the function body as before, but then `eval` the expanded function body to actually construct its closure. Then we fill in the dummy closure with this actual closure returned by `eval`.

To implement this optimization mechanism in our Lisp interpreter, we add an `if`-branch for the `p_define` special-form primitive to `expand`. This branch checks if we’re defining a new name with `T(v) == ATOM`, where `v = expand(car(x),e,b)` is an atom symbol if not associated with a primitive or a previously-defined global function. If this is the case, then we use a dummy closure `f = closure(nil,nil,nil)` to add to the temporarily extended environment `d = pair(v,f,e)` of environment `e`. To perform the next steps, we expand and evaluate the definition’s body `x` with `y = expand(x,d,b)` followed by `z = eval(y,e)`. If `z` is a closure, then we’re defining a function. This function is possibly a recursive function that invokes itself via the dummy `f`. We complete this dummy `f` with the resulting closure `z` to make all recursive self-references to `f` valid:

```
if (equ(f,p_define)) {
  v = expand(car(x),e,b);
  *p = cons(v,nil);
  p = cell+sp;
  x = car(cdr(x));
  if (T(v) == ATOM) {
    f = closure(nil,nil,nil);
    d = pair(v,f,e);
    y = expand(x,d,b);
    z = eval(y,e);
    if (T(z) == CLOS) {
```

```

        cell[ord(f)] = cdr(z);
        cell[ord(f)+1] = car(z);
        *p = cons(f,nil);
    }
    else *p = cons(y,nil);
}
else *p = cons(expand(x,e,b),nil);
return t;
}

```

Because recursive functions are prevalent in Lisp, this is a useful addition to complete our `expand` C function, after also declaring and adding `p_define = assoc(atom("define"),env)` to `main`.

Perhaps one more optimization could be performed. Local Lisp variables such as `d` and `n` in the last example are tracked in environment `e` in `expand`. Interpreting the code gives us information how deep these variables are located in the runtime environment when the code is evaluated by `eval`. Therefore, instead of calling `assoc` in `eval` we could look up a local variable using its depth from the top of the runtime environment, e.g. with a new cell type `LOCL` with a depth ordinal.

Leaving out garbage collection in this section highlights the logic without convoluting the code. However, without continuous garbage collection we lose memory cells with `expand` that won't be reused. On the other hand, runtime macro expansion also consumes additional memory cells that won't be reused without garbage collection.

17.2 Hygienic Macros

As we've seen before, plain dirty macros require obfuscation of locally-declared names in local scopes when macro arguments are used in the local scope. When macros use multiple local variables, each local must be obfuscated with a distinct name, making the macro more convoluted. Obfuscation prevents the *free variables* in the macro argument expressions to get *captured* by the locally-declared names. For example, `swap!` takes a variable `v` and value `x` to assign it to `v` while returning the old value of `v`:

```

(define swap!
  (macro (v x)
    `(let* (_ ,v)
      (progn
        (setq ,v ,x)
        _))))

```

The old value of `v` is stored in the local variable `_` to return it. However, obfuscation is fragile and it won't work to swap a variable named `_` with `(swap! _ k)`, since the macro-expanded code is:

```

(<let*> (_ _)
  (<progn>
    (<setq> _ k)
    _))

```

which does not assign the new value `k` to `_` in `(swap! _ k)`. Variable `_` is free (in the macro argument expression passed to variable `v`) and gets captured by the local scope of the local `_`.

Hygienic macros permit macro definitions that do not have naming conflicts. Hygienic macros do not need obfuscation. With hygienic macros we can sanitize the `swap!` macro and use a proper name for the local `let*` variable, say we pick `z` for example:

```

(define swap!
  (macro (v x)
    `(let* (z ,v)
      (progn
        (setq ,v ,x)
        z))))

```

Let's build a hygienic macro system. We want hygienic macros “straight out of the box”. We do not want to introduce new special forms or add new syntactic constructs in Lisp to support hygienic macros. The first step in our implementation is to recognize that free variables in macro argument expressions should not be captured in local scopes. We put these free variables “on hold”, symbolized by jail bars `|v|` holding a variable `v` to prevent it from being captured as a free variable. In other words, if we don't hold `|v|`, then it “steals” the identity of a local `v`. For example, the `(swap! z k)` macro-expanded code has a jailed `|z|` holding the `swap!` argument `z`:

```

(<let*> (z |z|)
  (<progn>
    (<setq> |z| k)
    z))

```

The `let*-local z` and `swap!` argument `|z|` are distinct. We can only safely release `|z|` by renaming the local `z` to something else, say `_z`. We can generate this new symbol `_z` when the `let*` is transformed into its final form:

```

(<let*> (_z z)
  (<progn>
    (<setq> z k)
    _z))

```

We use this hygienic macro mechanism to rename `lambda` argument variables and the local variables of `let*`, `let`, `letrec*`, and `letrec`. Renaming is only necessary when a naming conflict occurs. Implementing hygienic macros does not take much effort by augmenting the `expand` function to support them. We need a C function to hold all names (`ATOM` cells) that occur in a macro argument expression `x`, returning a new expression with `ATOM` cells replaced by `HOLD` cells:

```

L hold(L x) {
  if (T(x) == ATOM) return box(HOLD,ord(x));
  if (T(x) == CONS) {
    L t = nil,*p = &t;
    for (; T(x) == CONS; x = cdr(x),p = cell+sp) *p = cons(hold(car(x)),nil);
    *p = hold(x);
    return t;
  }
  return x;
}

```

where `HOLD` is a new type of cell that is similar to `ATOM`, but has no meaning outside of `expand` and its sub-functions, other than in `print` for execution tracing and for debug messaging purposes:

```

enum { ATOM = 0x7ff8,...,HOLD = 0x7ffe };
...
void print(L x) {
    if (T(x) == NIL) printf("");
    else if (T(x) == ATOM) printf("%s",A+ord(x));
    ...
    else if (T(x) == HOLD) printf("|%s|",A+ord(x));
    else printf("%.10lg",x);
}

```

To check whether an expression *x* holds a variable *v* as a **HOLD** cell somewhere, we define function **holds** that returns nonzero when *v* is being held in *x*:

```

I holds(L v,L x) {
    if (T(x) == HOLD) return ord(v) == ord(x);
    if (T(x) == CONS) return holds(v,car(x)) || holds(v,cdr(x));
    return 0;
}

```

For example, (**<progn>** (**<setq>** *|z|* *k*) *z*) holds *z* as *|z|*. After checking that an expression holds the name of a local variable, the local variable must be renamed with **gensym**:

```

L gensym(L v) { char sym[sizeof(buf)]; snprintf(sym,sizeof(sym),"%s",A+ord(v)); return atom(sym); }

```

Functions **holds** and **gensym** are used in function **hygienic**(*v*,*x*) to check, rename and return one new, if necessary, variable when *v* is an **ATOM** cell or a new list of (renamed) variables *w* when *v* is a **CONS** cell:

```

L hygienic(L v,L x) {
    if (T(v) == CONS) {
        L w = nil,*p = &w;
        for (; T(v) == CONS; v = cdr(v),p = cell+sp) *p = cons(hygienic(car(v),x),nil);
        if (T(v) == ATOM || T(v) == HOLD) *p = hygienic(v,x);
        return w;
    }
    if (T(v) == HOLD) return box(ATOM,ord(v));
    if (T(v) == ATOM) while (holds(v,x)) v = gensym(v);
    return v;
}

```

Note that **holds**(*v*,*x*) returns nonzero when *v* is held in expression *x*, which is used in a loop with **gensym** to find a suitably safe replacement name. As it is now safe to free all variables being held in an expression *x*, we define function **release** to do so:

```

L cede(L v) { return T(v) == HOLD ? box(ATOM,ord(v)) : v; }
L release(L x) {
    if (T(x) == CONS) {
        L *p = &x;
        for (; T(*p) == CONS; p = &cell[ord(*p)]) cell[ord(*p)+1] = release(car(*p));
        *p = release(*p);
    }
    return cede(x);
}

```

Function `release` destructively updates expression `x` by replacing `HOLD` cells with `ATOM` cells. It uses function `cede` to relinquish a `HOLD`. We can do this destructively because expression `x` is always constructed in macro expansion as new code by `expand`. It is not existing immutable program code. Finally, the `expand` function is updated as follows with a new branch on `HOLD`, an updated branch for macro application to `hold` macro argument expressions, and updated applications of special-form primitives `lambda`, `let*`, `let`, `letrec*`, and `letrec` to use `hygienic` and `release`:

```
L expand(L x,L e,L b) {
  L c,d,v,w,y;
  if (T(x) == ATOM) { ... }          /* case 1: the name of a variable */
  if (T(x) == HOLD) {
    x = release(x);
    if (lookup(x,e,&y) && (T(y) == PRIM || T(y) == CLOS || T(y) == MACR)) return y;
    return x;
  }
  if (T(x) == CLOS) { ... }          /* case 2: a function (closure) */
  if (T(x) == CONS) {
    ...
    if (T(f) == MACR) {              /* case 3: the application of a macro */
      for (c = nil,v = release(car(f)); T(v) == CONS; v = cdr(v),x = cdr(x))
        c = pair(car(v),quote(hold(car(x))),c);
      if (T(v) == ATOM) c = pair(v,quote(hold(x)),c);
      return expand(eval(expand(cdr(f),e,c),e),e,b);
    }
    if (T(f) == PRIM) {              /* case 4: the application of a special-form primitive */
      if (equ(f,p_quote)) { *p = release(x); return t; }
      if (equ(f,p_macro)) { *p = cons(car(x),cons(expand(car(cdr(x)),e,b),nil)); return t; }
      if (equ(f,p_lambda)) {
        v = car(x); w = hygienic(v,cdr(x));
        *p = cons(w,nil);
        for (d = e,c = b; T(v) == CONS; v = cdr(v),w = cdr(w))
          if (T(car(v)) == ATOM) d = pair(car(v),nil,d),c = pair(car(v),car(w),c);
          if (T(v) == ATOM) d = pair(v,nil,d),c = pair(v,w,c);
          cell[ord(*p)] = cons(expand(car(cdr(x)),d,c),nil);
          return t;
        }
      if (equ(f,p_cond)) { ... }
      if (equ(f,p_let*)) {
        for (d = e,c = b; let(x); x = cdr(x),p = &cell[ord(*p)]) {
          v = car(car(x)); w = hygienic(v,cdr(x));
          *p = cons(cons(w,cons(expand(car(cdr(car(x))),d,c),nil)),nil);
          if (T(v) == ATOM) d = pair(v,nil,d),c = pair(v,w,c);
        }
        *p = cons(expand(car(x),d,c),nil);
        return t;
      }
    }
    if (equ(f,p_let)) {
      for (d = e,c = b; let(x); x = cdr(x),p = &cell[ord(*p)]) {
        v = car(car(x)); w = hygienic(v,cdr(x));
        *p = cons(cons(w,cons(expand(car(cdr(car(x))),e,c),nil)),nil);
        if (T(v) == ATOM) d = pair(v,nil,d),c = pair(v,w,c);
      }
      *p = cons(expand(car(x),d,c),nil);
      return t;
    }
  }
}
```

```

if (equ(f,p_letreca)) {
  for (d = e, c = b; let(x); x = cdr(x), p = &cell[ord(*p)]) {
    v = car(car(x)); w = hygienic(v,x);
    if (T(v) == ATOM) d = pair(v,nil,d), c = pair(v,w,c);
    *p = cons(cons(w,cons(expand(car(cdr(car(x)))),d,c),nil)),nil);
  }
  *p = cons(expand(car(x),d,c),nil);
  return t;
}
if (equ(f,p_letrec)) {
  for (d = e, c = b, y = x; let(y); y = cdr(y)) {
    v = car(car(y)); w = hygienic(v,x);
    if (T(v) == ATOM) d = pair(v,nil,d), c = pair(v,w,c);
  }
  for (y = x; let(y); y = cdr(y), p = &cell[ord(*p)]) {
    v = car(car(y)); w = hygienic(v,x);
    *p = cons(cons(w,cons(expand(car(cdr(car(y)))),d,c),nil)),nil);
  }
  *p = cons(expand(car(y),d,c),nil);
  return t;
}
...
}
return x;
}

```

With early binding and hygienic macros added to our Lisp, we should remove the **MACR** branch from **eval**. Runtime macro expansion in **eval** is not hygienic. If macro expansion happens at runtime, then this is an error caused by a macro that is used before defined, including a recursive macro. Macros must be defined before being used. Recursive macros can be rewritten in non-recursive macros with a local **letrec*** code-constructing function, such as in the **case** macro for example.

We should update function **f_define** to avoid clobbering the global environment with non-**ATOM** names resulting from **expand** early binding names to primitives, closures and macros:

```

L f_define(L t, L *e) {
  L d = env, v = car(t);
  if (T(v) == PRIM) printf("not redefined built-in ");
  else if (T(v) != ATOM) {
    if (T(v) != CLOS && T(v) != MACR) return err(2,v);
    while (T(d) == CONS && !equ(v,cdr(car(d)))) d = cdr(d);
    if (T(d) != CONS) return err(2,v);
    v = car(car(d));
  }
  env = pair(v,eval(car(cdr(t)),*e),env);
  return v;
}

```

This permits global (re)definitions. But this eats up memory, since old definitions remain in memory. Old functions may also call other old functions instead of the redefined functions. When garbage collection is fully implemented, there is no problem to efficiently redefine functions and macros, i.e. redefinitions will efficiently replace old code with new code in cell memory.

Finally, we may have to **cede** a hold on an **ATOM** when a macro uses **eq?** to compare an atom given as a macro argument during macro expansion, which will be held. So we need to update:

```

L f_eq(L t, L *e) { I a = 0; L x = evarg(&t,e,&a); return equ(cede(x),cede(evarg(&t,e,&a))) ? tru : nil; }

```

18 Simple String Literals

Occasionally we may want our programs to display text, which we can do by printing a sequence of Lisp atoms that make up the words we want to display. It is simple enough to work, but cumbersome and tricky when we want to output text with spacing and punctuation. So let's add simple string support to our interpreter by extending the parser to accept `"`-quoted strings and store them as atoms. In this way, we can put anything in an atom except a `"`-quote. First, we change the `scan` function by adding an extra line of code when `seeing('\"')` in the input:

```
if (seeing('(') || ...) ...
else if (seeing('\"')) do buf[i++] = get(); while (i < sizeof(buf)-1 && (!seeing('\"') || !get()));
else do ...
```

The while-clause of the new line is a bit cryptic to keep things short. It just checks if we are not seeing the ending `"`-quote in the input. If we do see this, then we get it, discard it, and leave the loop by asserting that a `"`-quote code is indeed not zero with `!get()`. Second, we add a new line of code to the `parse` and also to the `tick` functions:

```
if (*buf == '\"') return cons(atom("quote"),cons(atom(buf+1),nil));
```

This line of code is almost identical to parsing and constructing a quoted atom, but we skip the leading `"`-quote stored in `buf`. This suffices to support `"`-quoted literal strings. To support longer strings, we should increase the size of `buf`. For example, change the declaration to `char buf[256]` or to even wider buffer sizes. When literal strings span multiple lines, then you should convert each newline (a zero byte we `see`) to a space for example:

```
char get() { char c = see ? see : ' '; look(); return c; }
```

If we want to define operations on strings, such as concatenation, slicing, and more, then we should create a new data type for strings rather than reusing the atom data type to store string literals, because atom allocation is slow to store atom text uniquely without creating duplicates.

19 Conclusions

The concepts and implementations presented in this article follow the original ideas and discoveries made by McCarthy in his 1960 paper. Lisp is an elegant programming language with simple primitives from which large and complex systems can be built. Many implementations of various flavors of Lisp and Scheme exist today. This article introduced and expanded a tiny Lisp interpreter. A fully-functional Lisp interpreter with 21 Lisp primitives, a REPL and simple garbage collection in only 99 lines of C. If you are adventurous to explore further, it should not be too difficult to expand the Lisp interpreter to support additional features. The door is open to experiment with alternative syntax and semantics for a Lisp hybrid or even a whole new language.

Any overlap or resemblance to any other Lisp implementations is coincidental. I wrote this article from scratch based on McCarthy's paper and my 20 years of experience teaching programming language courses that include Lisp/Scheme design and programming.

References

- [1] Graham, Paul (2002). “The Roots of Lisp” <http://www.paulgraham.com/rootsoflisp.html> retrieved July 9, 2022.
- [2] McCarthy, John. “Recursive functions of symbolic expressions and their computation by machine, part I.” *Communications of the ACM* 3.4 (1960): 184-195.
- [3] Church, Alonzo. “The calculi of lambda-conversion.” *Bull. Amer. Math. Soc* 50 (1944): 169-172.

A Tiny Lisp Interpreter with NaN boxing: 99 Lines of C

Lisp in 99 lines of C without comments:

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#define I unsigned
#define L double
#define A (char*)cell
#define N 1024
I hp=0,sp=N,ATOM=0x7ff8,PRIM=0x7ff9,CONS=0x7ffa,CLOS=0x7ffb,NIL=0x7ffc;
L cell[N],nil,tru,err,env;
I T(L x) { union { L x; unsigned long long i; } u = {x}; return u.i>>48; }
L box(I t,I i) { union { unsigned long long i; L x; } u = {(unsigned long long)t<<48|i}; return u.x; }
I ord(L x) { union { L x; unsigned long long i; } u = {x}; return u.i; }
L num(L n) { return n; }
I equ(L x,L y) { union { L x; unsigned long long i; } u = {x},v = {y}; return u.i == v.i; }
L atom(const char *s) {
  I i = 0; while (i < hp && strcmp(A+i,s)) i += strlen(A+i)+1;
  if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<3) abort();
  return box(ATOM,i);
}
L cons(L x,L y) { cell[--sp] = x; cell[--sp] = y; if (hp > sp<<3) abort(); return box(CONS,sp); }
L car(L p) { return (T(p)&^(CONS^CLOS)) == CONS ? cell[ord(p)+1] : err; }
L cdr(L p) { return (T(p)&^(CONS^CLOS)) == CONS ? cell[ord(p)] : err; }
L pair(L v,L x,L e) { return cons(cons(v,x),e); }
L closure(L v,L x,L e) { return box(CLOS,ord(pair(v,x,equ(e,env) ? nil : e))); }
L assoc(L v,L e) { while (T(e) == CONS && !equ(v,car(car(e)))) e = cdr(e); return T(e) == CONS ? cdr(car(e)) : err; }
I not(L x) { return T(x) == NIL; }
I let(L x) { return !not(x) && !not(cdr(x)); }
L eval(L,L),parse();
L evlis(L t,L e) { return T(t) == CONS ? cons(eval(car(t),e),evlis(cdr(t),e)) : T(t) == ATOM ? assoc(t,e) : nil; }
L f_eval(L t,L e) { return eval(car(evlis(t,e)),e); }
L f_quote(L t,L _) { return car(t); }
L f_cons(L t,L e) { return t = evlis(t,e),cons(car(t),car(cdr(t))); }
L f_car(L t,L e) { return car(car(evlis(t,e))); }
L f_cdr(L t,L e) { return cdr(car(evlis(t,e))); }
L f_add(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n += car(t); return num(n); }
L f_sub(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n -= car(t); return num(n); }
L f_mul(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n *= car(t); return num(n); }
L f_div(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n /= car(t); return num(n); }
L f_int(L t,L e) { L n = car(evlis(t,e)); return n < 1e16 && n > -1e16 ? (long long)n : n; }
L f_lt(L t,L e) { return t = evlis(t,e),car(t) - car(cdr(t)) < 0 ? tru : nil; }
L f_eq(L t,L e) { return t = evlis(t,e),equ(car(t),car(cdr(t))) ? tru : nil; }
L f_pair(L t,L e) { L x = car(evlis(t,e)); return T(x) == CONS ? tru : nil; }
L f_or(L t,L e) { L x = nil; while (!not(t) && not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_and(L t,L e) { L x = tru; while (!not(t) && !not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_not(L t,L e) { return not(car(evlis(t,e))) ? tru : nil; }
L f_cond(L t,L e) { while (not(eval(car(car(t))),e)) t = cdr(t); return eval(car(cdr(car(t))),e); }
L f_if(L t,L e) { return eval(car(cdr(not(eval(car(t),e)) ? cdr(t) : t)),e); }
L f_let(L t,L e) { for (;let(t); t = cdr(t)) e = pair(car(car(t)),eval(car(cdr(car(t))),e),e); return eval(car(t),e); }
L f_lambda(L t,L e) { return closure(car(t),car(cdr(t)),e); }
L f_define(L t,L e) { env = pair(car(t),eval(car(cdr(t)),e),env); return car(t); }
struct { const char *s; L (*f)(L,L); } prim[] = {
{"eval", f_eval }, {"car", f_car }, {"-", f_sub }, {"<", f_lt }, {"or", f_or }, {"cond", f_cond }, {"lambda", f_lambda },
{"quote", f_quote }, {"cdr", f_cdr }, {"*", f_mul }, {"int", f_int }, {"and", f_and }, {"if", f_if }, {"define", f_define },
{"cons", f_cons }, {"+", f_add }, {"/", f_div }, {"eq?", f_eq }, {"not", f_not }, {"let*", f_let }, {"pair?", f_pair }, {0};
L bind(L v,L t,L e) { return not(v) ? e : T(v) == CONS ? bind(cdr(v),cdr(t),pair(car(v),car(t),e)) : pair(v,t,e); }
L reduce(L f,L t,L e) { return eval(cdr(car(f)),bind(car(car(f)),evlis(t,e),not(cdr(f)) ? env : cdr(f))); }
L apply(L f,L t,L e) { return T(f) == PRIM ? prim[ord(f)].f(t,e) : T(f) == CLOS ? reduce(f,t,e) : err; }
L eval(L x,L e) { return T(x) == ATOM ? assoc(x,e) : T(x) == CONS ? apply(eval(car(x),e),cdr(x),e) : x; }
char buf[40],see = 0;
void look() { int c = getchar(); see = c; if (c == EOF) exit(0); }
```

```

I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : see == c; }
char get() { char c = see; look(); return c; }
char scan() {
    int i = 0;
    while (seeing(' ')) look();
    if (seeing('(') || seeing(')') || seeing('\')) buf[i++] = get();
    else do buf[i++] = get(); while (i < 39 && !seeing('(') && !seeing(')') && !seeing(' '));
    return buf[i] = 0,*buf;
}
L Read() { return scan(),parse(); }
L list() { L x; return scan() == ')' ? nil : !strcmp(buf, ".") ? (x = Read(),scan(),x) : (x = parse(),cons(x,list())); }
L quote() { return cons(atom("quote"),cons(Read(),nil)); }
L atomic() { L n; int i; return sscanf(buf,"%lg%n",&n,&i) > 0 && !buf[i] ? n : atom(buf); }
L parse() { return *buf == '(' ? list() : *buf == '\') ? quote() : atomic(); }
void print(L);
void printlist(L t) {
    for (putchar('('); ; putchar(' ')) {
        print(car(t));
        if (not(t = cdr(t))) break;
        if (T(t) != CONS) { printf(" . "); print(t); break; }
    }
    putchar(')');
}
void print(L x) {
    if (T(x) == NIL) printf("()");
    else if (T(x) == ATOM) printf("%s",A+ord(x));
    else if (T(x) == PRIM) printf("<%s>",prim[ord(x)].s);
    else if (T(x) == CONS) printlist(x);
    else if (T(x) == CLOS) printf("{%u}",ord(x));
    else printf("%.10lg",x);
}
void gc() { sp = ord(env); }
int main() {
    I i; printf("tinylisp");
    nil = box(NIL,0); err = atom("ERR"); tru = atom("#t"); env = pair(tru,tru,nil);
    for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
    while (1) { printf("\n%u>",sp-hp/8); print(eval(Read(),env)); gc(); }
}

```

B Tiny Lisp Interpreter with BCD boxing: 99 Lines of C

Lisp for the PC-G850 in 99 lines of C without comments:

```
#define I unsigned
#define L double
#define T *(char*)&
#define A (char*)cell
#define N 1024
I hp=0,sp=N,ATOM=32,PRIM=48,CONS=64,CLOS=80,NIL=96;
L cell[N],nil,tru,err,env;
L box(I t,I i) { L x = i+10; T x = t; return x; }
I ord(L x) { T x &= 15; return (I)x-10; }
L num(L n) { T n &= 159; return n; }
I equ(L x,L y) { return x == y; }
L atom(const char *s) {
  I i = 0; while (i < hp && strcmp(A+i,s)) i += strlen(A+i)+1;
  if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<3) abort();
  return box(ATOM,i);
}
L cons(L x,L y) { cell[--sp] = x; cell[--sp] = y; if (hp > sp<<3) abort(); return box(CONS,sp); }
L car(L p) { return (T p&^(CONS^CLOS)) == CONS ? cell[ord(p)+1] : err; }
L cdr(L p) { return (T p&^(CONS^CLOS)) == CONS ? cell[ord(p)] : err; }
L pair(L v,L x,L e) { return cons(cons(v,x),e); }
L closure(L v,L x,L e) { return box(CLOS,ord(pair(v,x,equ(e,env) ? nil : e))); }
L assoc(L v,L e) { while (T e == CONS && !equ(v,car(car(e)))) e = cdr(e); return T e == CONS ? cdr(car(e)) : err; }
I not(L x) { return T x == NIL; }
I let(L x) { return !not(x) && !not(cdr(x)); }
L eval(L,L),parse();
L evlis(L t,L e) { return T t == CONS ? cons(eval(car(t),e),evlis(cdr(t),e)) : T t == ATOM ? assoc(t,e) : nil; }
L f_eval(L t,L e) { return eval(car(evlis(t,e)),e); }
L f_quote(L t,L _) { return car(t); }
L f_cons(L t,L e) { return t = evlis(t,e),cons(car(t),car(cdr(t))); }
L f_car(L t,L e) { return car(car(evlis(t,e))); }
L f_cdr(L t,L e) { return cdr(car(evlis(t,e))); }
L f_add(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n += car(t); return num(n); }
L f_sub(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n -= car(t); return num(n); }
L f_mul(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n *= car(t); return num(n); }
L f_div(L t,L e) { L n = car(t = evlis(t,e)); while (!not(t = cdr(t))) n /= car(t); return num(n); }
L f_int(L t,L e) { L n = car(evlis(t,e)); return n-1e9 < 0 && n+1e9 > 0 ? (long)n : n; }
L f_lt(L t,L e) { return t = evlis(t,e),car(t) - car(cdr(t)) < 0 ? tru : nil; }
L f_eq(L t,L e) { return t = evlis(t,e),equ(car(t),car(cdr(t))) ? tru : nil; }
L f_or(L t,L e) { L x = nil; while (T t != NIL && not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_and(L t,L e) { L x = tru; while (T t != NIL && !not(x = eval(car(t),e))) t = cdr(t); return x; }
L f_not(L t,L e) { return not(car(evlis(t,e))) ? tru : nil; }
L f_cond(L t,L e) { while (not(eval(car(car(t)),e))) t = cdr(t); return eval(car(cdr(car(t))),e); }
L f_if(L t,L e) { return eval(car(cdr(not(eval(car(t),e)) ? cdr(t) : t)),e); }
L f_let(L t,L e) { for (;let(t); t = cdr(t)) e = pair(car(car(t)),eval(car(cdr(car(t))),e),e); return eval(car(t),e); }
L f_lambda(L t,L e) { return closure(car(t),car(cdr(t)),e); }
L f_define(L t,L e) { env = pair(car(t),eval(car(cdr(t)),e),env); return car(t); }
L f_pair(L t,L e) { L x = car(evlis(t,e)); return T x == CONS ? tru : nil; }
struct { const char *s; L (*f)(L,L); } prim[] = {
{"eval", f_eval },{"car",f_car},{ "-",f_sub},{ "<", f_lt },{"or", f_or },{"cond",f_cond},{ "lambda",f_lambda},
{"quote",f_quote},{ "cdr",f_cdr},{ "x*",f_mul},{ "int",f_int},{ "and",f_and},{ "if", f_if },{"define",f_define},
{"cons", f_cons },{"+", f_add},{ "/",f_div},{ "eq?",f_eq },{"not",f_not},{ "let*",f_let},{ "pair?", f_pair },{0}};
L bind(L v,L t,L e) { return T v == NIL ? e : T v == CONS ? bind(cdr(v),cdr(t),pair(car(v),car(t),e)) : pair(v,t,e); }
L reduce(L f,L t,L e) { return eval(cdr(car(f)),bind(car(car(f)),evlis(t,e),not(cdr(f)) ? env : cdr(f))); }
L apply(L f,L t,L e) { return T f == PRIM ? prim[ord(f)].f(t,e) : T f == CLOS ? reduce(f,t,e) : err; }
L eval(L x,L e) { return T x == ATOM ? assoc(x,e) : T x == CONS ? apply(eval(car(x),e),cdr(x),e) : x; }
char buf[40],see = 0;
void look() { int c = getchar(); see = c; if (c == -1) exit(0); }
I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : see == c; }
char get() { char c = see; look(); return c; }
char scan() {
```

```

int i = 0;
while (seeing(' ')) look();
if (seeing('(') || seeing(')') || seeing('\')) buf[i++] = get();
else do buf[i++] = get(); while (i < 39 && !seeing('(') && !seeing(')') && !seeing(' '));
return buf[i] = 0,*buf;
}
L read() { return scan(),parse(); }
L list() { L x; return scan() == ')' ? nil : !strcmp(buf, ".") ? (x = read(),scan(),x) : (x = parse(),cons(x,list())); }
L quote() { return cons(atom("quote"),cons(read(),nil)); }
L atomic() {
    L n; int i = strlen(buf);
    return isdigit(buf[*buf == '-']) && sscanf(buf,"%lg%n",&n,&i) && !buf[i] ? n : atom(buf);
}
L parse() { return *buf == '(' ? list() : *buf == '\'' ? quote() : atomic(); }
void print(L);
void printlist(L t) {
    for (putchar('('); ; putchar(' ')) {
        print(car(t));
        if (not(t = cdr(t))) break;
        if (T t != CONS) { printf(" . "); print(t); break; }
    }
    putchar(')');
}
void print(L x) {
    if (T x == NIL) printf("()");
    else if (T x == ATOM) printf("%s",A+ord(x));
    else if (T x == PRIM) printf("<%s>",prim[ord(x)].s);
    else if (T x == CONS) printlist(x);
    else if (T x == CLOS) printf("{%u}",ord(x));
    else printf("%.10lg",x);
}
void gc() { sp = ord(env); }
int main() {
    I i; printf("lisp850");
    nil = box(NIL,0); err = atom("ERR"); tru = atom("#t"); env = pair(tru,tru,nil);
    for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
    while (1) { printf("\n%u>",sp-hp/8); print(eval(read(),env)); gc(); }
}

```

C Optimized Lisp Interpreter with NaN boxing

Tail-call optimized Lisp interpreter for speed and reduced memory usage.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#define I unsigned
#define L double
#define A (char*)cell
#define N 1024
I hp=0,sp=N,ATOM=0x7ff8,PRIM=0x7ff9,CONS=0x7ffa,CLOS=0x7ffb,NIL=0x7ffc;
L cell[N],nil,tru,err,env;
I T(L x) { union { L x; unsigned long long i; } u = {x}; return u.i>>48; }
L box(I t,I i) { union { unsigned long long i; L x; } u = {(unsigned long long)t<<48|i}; return u.x; }
I ord(L x) { union { L x; unsigned long long i; } u = {x}; return u.i; }
L num(L n) { return n; }
I equ(L x,L y) { union { L x; unsigned long long i; } u = {x},v = {y}; return u.i == v.i; }
L atom(const char *s) {
  I i = 0; while (i < hp && strcmp(A+i,s)) i += strlen(A+i)+1;
  if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<3) abort();
  return box(ATOM,i);
}
L cons(L x,L y) { cell[--sp] = x; cell[--sp] = y; if (hp > sp<<3) abort(); return box(CONS,sp); }
L car(L p) { return (T(p)&^(CONS^CLOS)) == CONS ? cell[ord(p)+1] : err; }
L cdr(L p) { return (T(p)&^(CONS^CLOS)) == CONS ? cell[ord(p)] : err; }
L pair(L v,L x,L e) { return cons(cons(v,x),e); }
L closure(L v,L x,L e) { return box(CLOS,ord(pair(v,x,equ(e,env) ? nil : e))); }
L assoc(L v,L e) { while (T(e) == CONS && !equ(v,car(car(e)))) e = cdr(e); return T(e) == CONS ? cdr(car(e)) : err; }
I not(L x) { return T(x) == NIL; }
I let(L x) { return !not(x) && !not(cdr(x)); }
L eval(L,L),parse();
L evlis(L t,L e) {
  L s,*p;
  for (s = nil,p = &s; T(t) == CONS; p = cell+sp,t = cdr(t)) *p = cons(eval(car(t),e),nil);
  if (T(t) == ATOM) *p = assoc(t,e);
  return s;
}
L evarg(L *t,L *e,I *a) {
  L x;
  if (T(*t) == ATOM && !*a) *t = assoc(*t,*e),*a = 1;
  x = car(*t); *t = cdr(*t);
  return *a ? x : eval(x,*e);
}
I isarg(L *t,L *e,I *a,L *x) {
  if (T(*t) == ATOM && !*a) *t = assoc(*t,*e),*a = 1;
  if (T(*t) != CONS) return 0;
  *x = car(*t); *t = cdr(*t);
  if (!*a) *x = eval(*x,*e);
  return 1;
}
L f_eval(L t,L *e) { I a = 0; return evarg(&t,e,&a); }
L f_quote(L t,L *) { return car(t); }
L f_cons(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return cons(x,evarg(&t,e,&a)); }
L f_car(L t,L *e) { I a = 0; return car(evarg(&t,e,&a)); }
L f_cdr(L t,L *e) { I a = 0; return cdr(evarg(&t,e,&a)); }
L f_add(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n += x; return num(n); }
L f_sub(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n -= x; return num(n); }
L f_mul(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n *= x; return num(n); }
L f_div(L t,L *e) { I a = 0; L x,n = evarg(&t,e,&a); while (isarg(&t,e,&a,&x)) n /= x; return num(n); }
L f_int(L t,L *e) { I a = 0; L n = evarg(&t,e,&a); return n < 1e16 && n > -1e16 ? (long long)n : n; }
L f_lt(L t,L *e) { I a = 0; L n = evarg(&t,e,&a); return n - evarg(&t,e,&a) < 0 ? tru : nil; }
L f_eq(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return equ(x,evarg(&t,e,&a)) ? tru : nil; }
L f_pair(L t,L *e) { I a = 0; L x = evarg(&t,e,&a); return T(x) == CONS ? tru : nil; }
```

```

L f_not(L t,L *e) { I a = 0; return not(evarg(&t,e,&a)) ? tru : nil; }
L f_or(L t,L *e) { I a = 0; L x = nil; while (isarg(&t,e,&a,&x) && not(x)) continue; return x; }
L f_and(L t,L *e) { I a = 0; L x = tru; while (isarg(&t,e,&a,&x) && !not(x)) continue; return x; }
L f_cond(L t,L *e) { while (not(eval(car(car(t))),*e)) t = cdr(t); return car(cdr(car(t))); }
L f_if(L t,L *e) { return car(cdr(not(eval(car(t),*e)) ? cdr(t) : t)); }
L f_let(L t,L *e) { for (; let(t); t = cdr(t)) *e = pair(car(car(t)),eval(car(cdr(car(t))),*e),*e); return car(t); }
L f_lambda(L t,L *e) { return closure(car(t),car(cdr(t)),*e); }
L f_define(L t,L *e) { env = pair(car(t),eval(car(cdr(t)),*e),env); return car(t); }
struct { const char *s; L (*f)(L,L*); short t; } prim[] = {
{"eval", f_eval, 1},{ "quote", f_quote, 0},{ "cons", f_cons, 0},{ "car", f_car, 0},{ "cdr", f_cdr, 0},{ "+", f_add, 0},
{"-", f_sub, 0},{ "*", f_mul, 0},{ "/", f_div, 0},{ "int", f_int, 0},{ "<", f_lt, 0},{ "eq?", f_eq, 0},
{"or", f_or, 0},{ "and", f_and, 0},{ "not", f_not, 0},{ "cond", f_cond, 1},{ "if", f_if, 1},{ "let*", f_let, 1},
{"lambda", f_lambda, 0},{ "define", f_define, 0},{ "pair?", f_pair, 0},{ 0};
void assign(L v,L x,L e) { while (!equ(v,car(car(e)))) e = cdr(e); cell[ord(car(e))] = x; }
L eval(L x,L e) {
I a; L f,v,d,g = nil,h;
while (1) {
if (T(x) == ATOM) return assoc(x,e);
if (T(x) != CONS) return x;
f = eval(car(x),e); x = cdr(x);
if (T(f) == PRIM) {
x = prim[ord(f)].f(x,&e);
if (prim[ord(f)].t) continue;
return x;
}
if (T(f) != CLOS) return err;
v = car(car(f));
if (equ(f,g)) d = e;
else if (not(d = cdr(f))) d = env;
for (a = 0; T(v) == CONS; v = cdr(v)) d = pair(car(v),evarg(&x,&e,&a),d);
if (T(v) == ATOM) d = pair(v,a ? x : evlis(x,e),d);
if (equ(f,g)) {
for (; !equ(d,e) && sp == ord(d); d = cdr(d),sp += 4) assign(car(car(d)),cdr(car(d)),e);
for (; !equ(d,h) && sp == ord(d); d = cdr(d)) sp += 4;
}
x = cdr(car(f)); e = d; g = f; h = e;
}
}
char buf[40],see = 0;
void look() { int c = getchar(); see = c; if (c == EOF) exit(0); }
I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : see == c; }
char get() { char c = see; look(); return c; }
char scan() {
int i = 0;
while (seeing(' ')) look();
if (seeing('(') || seeing(')') || seeing('\n')) buf[i++] = get();
else do buf[i++] = get(); while (i < 39 && !seeing('(') && !seeing(')') && !seeing('\n'));
return buf[i] = 0,*buf;
}
L Read() { return scan(),parse(); }
L list() {
L t,*p;
for (t = nil,p = &t; ; *p = cons(parse(),nil),p = cell+sp) {
if (scan() == ')') return t;
if (*buf == '.' && !buf[1]) return *p = Read(),scan(),t;
}
}
L parse() {
L n; int i;
if (*buf == '(') return list();
if (*buf == '\\') return cons(atom("quote"),cons(Read(),nil));
return sscanf(buf,"%lg%n",&n,&i) > 0 && !buf[i] ? n : atom(buf);
}
void print(L);
void printlist(L t) {

```

```

for (putchar('('); ; putchar(' ')) {
    print(car(t));
    if (not(t = cdr(t))) break;
    if (T(t) != CONS) { printf(" . "); print(t); break; }
}
putchar(')');
}

void print(L x) {
    if (T(x) == NIL) printf("()");
    else if (T(x) == ATOM) printf("%s",A+ord(x));
    else if (T(x) == PRIM) printf("<%s>",prim[ord(x)].s);
    else if (T(x) == CONS) printlist(x);
    else if (T(x) == CLOS) printf("{%u}",ord(x));
    else printf("%.10lg",x);
}

void gc() { sp = ord(env); }

int main() {
    I i; printf("tinylisp");
    nil = box(NIL,0); err = atom("ERR"); tru = atom("#t"); env = pair(tru,tru,nil);
    for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
    while (1) { printf("\n%u>",sp-hp/8); print(eval(Read(),env)); gc(); }
}

```

D Optimized Lisp Interpreter with BCD boxing

The following version of the Lisp interpreter for the PC-G850 is minimally optimized to keep the code size at a minimum.

```
#define I unsigned
#define L double
#define T *(char*)&
#define A (char*)cell
#define N 1024
I hp=0,sp=N,ATOM=32,PRIM=48,CONS=64,CLOS=80,NIL=96;
L cell[N],nil,tru,err,env;
L box(I t,I i) { L x = i+10; T x = t; return x; }
I ord(L x) { T x &= 15; return (I)x-10; }
L num(L n) { T n &= 159; return n; }
L atom(const char *s) {
  I i = 0; while (i < hp && strcmp(A+i,s)) i += strlen(A+i)+1;
  if (i == hp && (hp += strlen(strcpy(A+i,s))+1) > sp<<3) abort();
  return box(ATOM,i);
}
L cons(L x,L y) { cell[--sp] = x; cell[--sp] = y; if (hp > sp<<3) abort(); return box(CONS,sp); }
L car(L p) { return (T p&224) == CONS ? cell[T p &= 15,(I)p-9] : err; }
L cdr(L p) { return (T p&224) == CONS ? cell[T p &= 15,(I)p-10] : err; }
L pair(L v,L x,L e) { return cons(cons(v,x),e); }
L closure(L v,L x,L e) { return box(CLOS,ord(pair(v,x,e == env ? nil : e))); }
L assoc(L v,L e) { while (T e == CONS && v != car(car(e))) e = cdr(e); return T e == CONS ? cdr(car(e)) : err; }
I not(L x) { return T x == NIL; }
I let(L x) { return !not(x) && !not(cdr(x)); }
L eval(L,L),parse();
L evlis(L t,L e) {
  L s,*p;
  for (s = nil,p = &s; T t == CONS; p = cell+sp,t = cdr(t)) *p = cons(eval(car(t),e),nil);
  if (T t == ATOM) *p = assoc(t,e);
  return s;
}
L f_eval(L t,L *e) { return car(evlis(t,*e)); }
L f_quote(L t,L *_) { return car(t); }
L f_cons(L t,L *e) { return t = evlis(t,*e),cons(car(t),car(cdr(t))); }
L f_car(L t,L *e) { return car(car(evlis(t,*e))); }
L f_cdr(L t,L *e) { return cdr(car(evlis(t,*e))); }
L f_add(L t,L *e) { L n = car(t = evlis(t,*e)); while (!not(t = cdr(t))) n += car(t); return num(n); }
L f_sub(L t,L *e) { L n = car(t = evlis(t,*e)); while (!not(t = cdr(t))) n -= car(t); return num(n); }
L f_mul(L t,L *e) { L n = car(t = evlis(t,*e)); while (!not(t = cdr(t))) n *= car(t); return num(n); }
L f_div(L t,L *e) { L n = car(t = evlis(t,*e)); while (!not(t = cdr(t))) n /= car(t); return num(n); }
L f_int(L t,L *e) { L n = car(evlis(t,*e)); return n-1e9 < 0 && n+1e9 > 0 ? (long)n : n; }
L f_lt(L t,L *e) { return t = evlis(t,*e),car(t) - car(cdr(t)) < 0 ? tru : nil; }
L f_eq(L t,L *e) { return t = evlis(t,*e),car(t) == car(cdr(t)) ? tru : nil; }
L f_pair(L t,L *e) { L x = car(evlis(t,*e)); return T x == CONS ? tru : nil; }
L f_or(L t,L *e) { I a = 0; L x = nil; while (isarg(&t,e,&a,&x) && not(x)) continue; return x; }
L f_and(L t,L *e) { I a = 0; L x = tru; while (isarg(&t,e,&a,&x) && !not(x)) continue; return x; }
L f_not(L t,L *e) { return not(car(evlis(t,*e))) ? tru : nil; }
L f_cond(L t,L *e) { while (not(eval(car(car(t)),*e))) t = cdr(t); return car(cdr(car(t))); }
L f_if(L t,L *e) { return car(cdr(not(eval(car(t),*e)) ? cdr(t) : t)); }
L f_let(L t,L *e) { for (;let(t); t = cdr(t)) *e = pair(car(car(t)),eval(car(cdr(car(t))),*e),*e); return car(t); }
L f_lambda(L t,L *e) { return closure(car(t),car(cdr(t)),*e); }
L f_define(L t,L *e) { env = pair(car(t),eval(car(cdr(t)),*e),env); return car(t); }
struct { const char *s; L (*f)(L,L*); short t; } prim[] = {
{"eval", f_eval, 1},{ "quote", f_quote, 0},{ "cons", f_cons, 0},{ "car", f_car, 0},{ "cdr", f_cdr, 0},{ "+", f_add, 0},
{"-", f_sub, 0},{ "*", f_mul, 0},{ "/", f_div, 0},{ "int", f_int, 0},{ "<", f_lt, 0},{ "eq?", f_eq, 0},
{"or", f_or, 0},{ "and", f_and, 0},{ "not", f_not, 0},{ "cond", f_cond, 1},{ "if", f_if, 1},{ "let*", f_let, 1},
{"lambda", f_lambda, 0},{ "define", f_define, 0},{ "pair?", f_pair, 0},{ 0};
L eval(L x,L e) {
  L f,v,d;
  while (1) {
```

```

    if (T x == ATOM) return assoc(x,e);
    if (T x != CONS) return x;
    f = eval(car(x),e); x = cdr(x);
    if (T f == PRIM) {
        x = prim[ord(f)].f(x,&e);
        if (prim[ord(f)].t) continue;
        return x;
    }
    if (T f != CLOS) return err;
    v = car(car(f)); d = cdr(f);
    if (T d == NIL) d = env;
    for (;T v == CONS && T x == CONS; v = cdr(v),x = cdr(x)) d = pair(car(v),eval(car(x),e),d);
    for (x = T x == CONS ? evalis(x,e) : eval(x,e); T v == CONS; v = cdr(v),x = cdr(x)) d = pair(car(v),car(x),d);
    if (T v == ATOM) d = pair(v,x,d);
    x = cdr(car(f)); e = d;
}
}
char buf[40],see = 0;
void look() { int c = getchar(); see = c; if (c == -1) exit(0); }
I seeing(char c) { return c == ' ' ? see >= 0 && see <= c : see == c; }
char get() { char c = see; look(); return c; }
char scan() {
    int i = 0;
    while (seeing(' ')) look();
    if (seeing('(') || seeing(')') || seeing('\')) buf[i++] = get();
    else do buf[i++] = get(); while (i < 39 && !seeing('(') && !seeing(')') && !seeing('\'));
    return buf[i] = 0,*buf;
}
L read() { return scan(),parse(); }
L list() {
    L t,*p;
    for (t = nil,p = &t; ; *p = cons(parse(),nil),p = cell+sp) {
        if (scan() == ')') return t;
        if (*buf == '.' && !buf[1]) return *p = read(),scan(),t;
    }
}
L parse() {
    L n; int i;
    if (*buf == '(') return list();
    if (*buf == '\') return cons(atom("quote"),cons(read(),nil));
    i = strlen(buf);
    return isdigit(buf[*buf == '-']) && sscanf(buf,"%lg%n",&n,&i) > 0 && !buf[i] ? n : atom(buf);
}
void print(L);
void printlist(L t) {
    for (putchar('('); ; putchar(' ')) {
        print(car(t));
        if (not(t = cdr(t))) break;
        if (T t != CONS) { printf(" . "); print(t); break; }
    }
    putchar(')');
}
void print(L x) {
    if (T x == NIL) printf("()");
    else if (T x == ATOM) printf("%s",A+ord(x));
    else if (T x == PRIM) printf("<%s>",prim[ord(x)].s);
    else if (T x == CONS) printlist(x);
    else if (T x == CLOS) printf("{%u}",ord(x));
    else printf("%.10lg",x);
}
void gc() { sp = ord(env); }
int main() {
    int i;
    printf("lisp850");
    nil = box(NIL,0); err = atom("ERR"); tru = atom("#t"); env = pair(tru,tru,nil);

```

```
for (i = 0; prim[i].s; ++i) env = pair(atom(prim[i].s),box(PRIM,i),env);
while (1) { printf("\n%u>",sp-hp/8); print(eval(read(),env)); gc(); }
}
```

E Example Lisp Functions and Macros

E.1 Standard Lisp Functions

The following functions should be self-explanatory, for details see further below:

```
(define null? not)
(define err? (lambda (x) (eq? x 'ERR)))
(define number? (lambda (x) (eq? (* 0 x) 0)))
(define symbol?
  (lambda (x)
    (and
      x
      (not (number? x))
      (not (pair? x)))))
(define atom?
  (lambda (x)
    (or
      (not x)
      (symbol? x))))
(define list?
  (lambda (x)
    (if (not x)
        #t
        (if (pair? x)
            (list? (cdr x))
            ())))))
(define equal?
  (lambda (x y)
    (or
      (eq? x y)
      (and
        (pair? x)
        (pair? y)
        (equal? (car x) (car y))
        (equal? (cdr x) (cdr y))))))
(define neg (lambda (n) (- 0 n)))
(define > (lambda (x y) (< y x)))
(define <= (lambda (x y) (not (< y x))))
(define >= (lambda (x y) (not (< x y))))
(define = (lambda (x y) (eq? (- x y) 0)))
(define list (lambda args args))
(define cadr (lambda (x) (car (cdr x))))
(define caddr (lambda (x) (car (cdr (cdr x)))))
(define begin (lambda (x . args) (if args (begin . args) x)))
```

Explanation:

- `null?` tests for an empty list.
- `err?` tests if a value is `ERR`.
- `equal?` tests equality of two values recursively over lists (`eq?` tests exact equality only).
- `symbol?` tests if the value is an atom excluding the empty list.

- `atom?` tests if the value is an atom including the empty list, but beware that some Lisp implementation also return `#t` for numbers.
- `list` returns a list of the values of the arguments. For example, `(list 1 2 (+ 1 2))` gives `(1 2 3)`.
- `begin` (called `progn` in some other Lisp) returns the last value of its last argument. For example, `(begin 1 2 (+ 1 2))` gives 3. This function is often used as a code block in Lisp, to evaluate a sequence of expressions, which only makes sense if the expressions have side effects, such as `setq` to change the value of a variable. This variation of `begin` does not work without arguments passed to `begin`, which is a useless use case anyway.

E.2 Math Functions

The following functions should be self-explanatory:

```
(define abs
  (lambda (n)
    (if (< n 0)
        (- 0 n)
        n)))
(define frac (lambda (n) (- n (int n))))
(define truncate int)
(define floor
  (lambda (n)
    (int
     (if (< n 0)
         (- n 1)
         n))))
(define ceiling (lambda (n) (- 0 (floor (- 0 n)))))
(define round (lambda (n) (floor (+ n 0.5))))
(define mod (lambda (n m) (- n (* m (int (/ n m))))))
(define gcd
  (lambda (n m)
    (if (eq? m 0)
        n
        (gcd m (mod n m)))))
(define lcm (lambda (n m) (/ (* n m) (gcd n m))))
(define even? (lambda (n) (eq? (mod n 2) 0)))
(define odd? (lambda (n) (eq? (mod n 2) 1)))
```

E.3 List Functions

The following functions should be self-explanatory, for details see further below:

```
(define length-tr
  (lambda (t n)
    (if t
        (length-tr (cdr t) (+ n 1))
        n)))
(define length (lambda (t) (length-tr t 0)))
(define append1
  (lambda (s t)
    (if s
```

```

        (cons (car s) (append1 (cdr s) t))
      t)))
(define append
  (lambda (t . args)
    (if args
      (append1 t (append . args))
      t)))
(define nthcdr
  (lambda (n t)
    (if (< 0 n)
      (nthcdr (- n 1) (cdr t))
      t)))
(define nth (lambda (n t) (car (nthcdr n t))))
(define reverse-tr
  (lambda (r t)
    (if t
      (reverse-tr (cons (car t) r) (cdr t))
      r)))
(define reverse (lambda (t) (reverse-tr () t)))
(define last-tr
  (lambda (t n s)
    (if t
      (if (< n 1)
        (last-tr (cdr t) n (cdr s))
        (last-tr (cdr t) (- n 1) s))
      s)))
(define last (lambda (t . n) (last-tr t (if n (car n) 1) t)))
(define member
  (lambda (x t)
    (if t
      (if (equal? x (car t))
        t
        (member x (cdr t)))
      t)))
(define foldr
  (lambda (f x t)
    (if t
      (f (car t) (foldr f x (cdr t)))
      x)))
(define foldl
  (lambda (f x t)
    (if t
      (foldl f (f (car t) x) (cdr t))
      x)))
(define min
  (lambda args
    (foldl
      (lambda (x y) (if (< x y) x y))
      inf
      args)))
(define max
  (lambda args
    (foldl (lambda (x y) (if (< x y) y x))
      -inf
      args)))
(define filter

```

```

(lambda (f t)
  (if t
    (if (f (car t))
      (cons (car t) (filter f (cdr t)))
      (filter f (cdr t)))
    ())))

(define all?
  (lambda (f t)
    (if t
      (if (f (car t))
        (all? f (cdr t))
        ())
      #t)))

(define any?
  (lambda (f t)
    (if t
      (if (f (car t))
        #t
        (any? f (cdr t)))
      ())))

(define mapcar
  (lambda (f t)
    (if t
      (cons (f (car t)) (mapcar f (cdr t)))
      ())))

(define map
  (lambda (f . args)
    (if (any? not args)
      ()
      (let*
        (x (mapcar car args))
        (t (mapcar cdr args))
        (cons (f . x) (map f . t))))))

(define zip (lambda args (map list . args)))

(define seq-prepend
  (lambda (t n m)
    (if (< n m)
      (seq-prepend (cons (- m 1) t) n (- m 1))
      t)))

(define seq (lambda (n m) (seq-prepend () n m)))

(define seqby-prepend
  (lambda (t n m k)
    (if (< 0 (* k (- m n)))
      (seqby-prepend (cons (- m k) t) n (- m k) k)
      t)))

(define seqby (lambda (n m k) (seqby-prepend () n (+ n k (* k (int (/ (- m n) (if (< 0 k) 1 -1)) k)))) k)))

(define range
  (lambda (n m . args)
    (if args
      (seqby n m (car args))
      (seq n m))))

(define copy-list
  (lambda (t)
    (if (pair? t)
      (cons (car t) (copy-list (cdr t)))
      t)))

```

```

(define make-list-tr
  (lambda (n x t)
    (if (< 0 n)
        (make-list-tr (- n 1) x (cons x t))
        t)))
(define make-list (lambda (n . x) (make-list-tr n (if x (car x) ()) ())))

```

Explanation:

- **length** returns the length of list t using tail-recursive **length-tr**.
- **append** returns the concatenation of multiple lists.
- **nthcdr** skips n elements of list t to return the n 'th cdr (tail recursive).
- **nth** returns the n 'th element of list t (tail recursive).
- **reverse** reverses list t using tail-recursive **reverse-tr**.
- **last** returns the last n element(s) of list t or $()$ when the list is empty, optional $n = 1$ by default (tail recursive).
- **member** checks list membership and returns the rest of list t where x was found (tail recursive).
- **foldr** and **foldl** return the value of right- and left-folded list t using an operator $f = \oplus$ and initial value x : **(foldl** $\oplus$ x_0 '(x_1 x_2 ... x_n)) = $(\cdots((x_0 \oplus x_1) \oplus x_2) \oplus \cdots x_n)$ and **(foldr** $\oplus$ x_0 '(x_1 x_2 ... x_n)) = $(x_1 \oplus (x_2 \oplus (\cdots (x_n \oplus x_0))))$ (**foldl** is tail recursive, **foldr** is not).
- **min** and **max** return the minimum and maximum value of their arguments, respectively, using tail-recursive **foldl**, note that **inf** and **-inf** are standard IEEE-754 (double) float infinities.
- **filter** returns a list with elements x from list t for which $(f\ x)$ is true (i.e. not false $()$).
- **all?** returns **#t** if all elements x of the list t satisfy $(f\ x)$, returning $()$ otherwise (tail recursive).
- **any?** returns **#t** if any one of the elements x of the list t satisfy $(f\ x)$, returning $()$ otherwise (tail recursive).
- **mapcar** applies f to each of the elements of a list t .
- **map** applies f to the elements of a list and for n lists for n -ary function f .
- **zip** takes n lists of length m to return a list of length m with lists of length n .
- **seq-prepend** generates a list of successive values n up to but not including m and prepends this to the list t (tail recursive)
- **seq** generates a list of successive values n up to but not including m using tail-recursive **seq-prepend**.
- **seqby-prepend** generates a list of values from n to but not including m stepping by k and prepends this to the list t (tail recursive)

- `seqby` generates a list of values from n to but not including m stepping by k using tail-recursive `seqby-prepend`.
- `range` generates a list of values from n to but not including m with an optional step value using `seq` and `seqby`.
- `copy-list` returns a copy of a list, which may be useful when making destructive updates to a list with `set-car!` and `set-cdr!` when updates should not be shared by other lists.
- `make-list` generates a list of n copies of x , where x is `()` by default (tail recursive).

E.4 Other Higher-Order Functions

The following functions take one or more functions as arguments to construct new functions:

```
(define curry (lambda (f x) (lambda args (f x . args))))
(define compose (lambda (f g) (lambda args (f (g . args)))))
(define complement (lambda (f) (lambda args (not (f . args)))))
(define Y (lambda (f) (lambda args ((f (Y f)) . args)))))
```

Explanation:

- `curry` takes a function `f` and an argument `x` and returns a function that applies `f` to `x` and the given arguments.
- `compose` takes two function `f` and `g` and returns a function that applies `g` to the arguments followed by the application of `f` to the result. For example, `(define div (compose int /))` defines an integer-truncating division `div` function such that `(div 7 3)` returns 2.
- `complement` takes a function `f` to return a function that when applied to one or more arguments returns the complement Boolean of `f`, i.e. returns `()` (false) when `f` returns `#t` (true) and vice versa.
- `Y` is the *Y combinator* that takes a function `f` to return a function that applies `f` to `(Y f)` that is a copy of itself, and in turn returns a self-applying (recursive) function. The `Y` combinator can be used for recursion without naming the function. For example the factorial of 5 is 120:

```
> ((Y (lambda (f) (lambda (k) (if (< 1 k) (* k (f (- k 1))) 1)))) 5)
120
```

E.5 Revealing Closures

The following function reveals the arguments and body of the closure of a lambda:

```
(define reveal (lambda (f) (cons 'lambda (cons (car (car f)) (cons (cdr (car f)) ())))))
```

Explanation:

- `reveal` takes a closure and “unparses” it as a lambda. For example `(reveal reveal)` shows `(lambda (f) (cons (quote lambda) (cons (car (car f)) (cons (cdr (car f)) ())))).`

E.6 Macros

The `defmacro` and `defun` macros are typically found in other Lisp. The `apply` macro is a built-in function in other Lisp, whereas our Lisp can do the same with a macro that translates to an application with the dot operator, as explained further below:

```
(define defmacro
  (macro (f v x)
    `(define ,f (macro ,v ,x))))
(defmacro defun (f v x)
  `(define ,f (lambda ,v ,x)))
(defmacro when (x . args) `(if ,x (progn . ,args) ()))
(defmacro unless (x . args) `(if ,x () (progn . ,args)))
(defmacro apply fargs
  (letrec*
    (last (lambda (t)
      (if (cdr t)
        (last (cdr t))
        t)))
    (app (lambda (t s)
      (if (cdr t)
        (cons (car t) (app (cdr t) s))
        s)))
    `(let* (_ ,(car (last fargs))) ,(app fargs '_))))
(defmacro case (x . args)
  (letrec*
    (cc (lambda (t)
      (if t
        (if (eq? (car (car t)) 'otherwise)
          `((#t ,(car (cdr (car t)))))
          `(((eq? _ ',(car (car t))) ,(car (cdr (car t)))) . ,(cc (cdr t))))
          `((#t ())))))
    `(let* (_ ,x) (cond . ,(cc args)))))
(defmacro dolist (x . args)
  `(let*
    (,(car x) ())
    (_ ,(car (cdr x)))
    (while _
      (setq ,(car x) (car _))
      (setq _ (cdr _))
      . ,args)))
(defmacro for (x . args)
  (let*
    (v (car x))
    (a (car (cdr x)))
    (b (car (cdr (cdr x))))
    (if (cdr (cdr (cdr x)))
      (let* (s (car (cdr (cdr (cdr x)))))
        `(let* (,v (- ,a ,s)) (_ (* ,s ,b))
          (while (not (< _ (* ,s (setq ,v (+ ,v ,s))))) . ,args)))
        `(let* (,v (- ,a 1)) (_ (+ ,b 1))
          (while (< (setq ,v (+ ,v 1)) _) . ,args)))))
```

Explanation:

- **defmacro** takes the name of a new macro **f**, its list of arguments **v**, and macro body **x** to define a new macro.
- **defun** takes the name of a new function **f**, its list of arguments **v**, and function body **x** to define a new function.
- **when** takes a condition and one or more expressions to evaluate if the condition evaluates to true (i.e. not **()**).
- **unless** takes a condition and one or more expressions to evaluate if the condition evaluates to false (i.e. **()**).
- **apply** takes a function with its arguments and treats the last argument as an argument list. For example, `(apply + 1 2 '(3 4))` translates to `(let* (_ '(3 4)) (+ 1 2 . _))`, using the dot operator to apply more arguments given by the name of a variable associated with a list, which in this case evaluates to 10.
- **case** is an extended version of the **case** macro defined in Section 17, where this **case** has constant keys (i.e. they're quoted) and may have an optional last **otherwise** clause. It uses a local recursive code-compiling function **cc** to construct `(let* (_ x) (cond ((eq? _ 'y1) x1) ... ((eq? _ 'yk) xk)))` for the *k* keys *y_i* and corresponding expression *x_i* to be evaluated when the key matches.
- **dolist** takes a tuple `(v t)` with variable *v* and a list *t* to iterate *v* over the list *t* values while executing one or more expressions given as the next arguments to **dolist**. For example, `(dolist (k '(7 11 42)) (println (* k k)))` prints the squares of 7, 11 and 42. Note that **mapcar** or **filter** work just as well without side-effects (i.e. they are *pure* higher-order functions), e.g. `(filter (lambda (x) (println (* x x))) '(7 11 42))`.
- **for** is a modified and extended version of the **for** macro defined in Section 16.6. It takes a list `(v a b s)` with a loop counter variable *v*, a starting value *a*, an ending value *b* and optional step *s* (default *s* = 1) to construct an iterative loop over one or more expressions given as the next arguments to **for**. For example `(for (i 1 10) (println i))` prints 1 to 10. Note that **mapcar** and **filter** with **seq** or **range** work just as well without side-effects (i.e. they are *pure* higher-order functions), e.g. `(filter println (seq 1 11))`.

Note that some macro definitions use *obfuscation* with an “anonymous” variable named **_** to prevent accidental macro argument capture. Obfuscation is not necessary with *hygienic macros* described in Section 17.2.

F List of Additions, Edits and Corrections

- improved `eval` in section "Tail-call optimization" to handle the dot operator when occurring anywhere in lists of formal and actual arguments (one case failed)
- add `pair?` as a new built-in primitive instead of externally defined
- add improved `evarg` to replace `arg` in section "Optimizing the Lisp Primitives"
- new section "Catching SIGINT to Stop Running Programs"
- fix undefined behavior in C in the `letrec*` primitive in section "Additional Lisp Primitives"
- update section "Optimizing the Lisp primitives" to include execution tracing and dumping
- new section "Tail-calls à trois"
- update Exhibit E with tail-recursive versions of Lisp definitions when applicable
- new `letrec` primitive in section "Additional Lisp Primitives"
- new backquote and comma operators in section "Additional Lisp Primitives" with `macro`
- minor improvement of section "Adding Error Handling and Exceptions" to define `err()` to show the cause of an error when tracing Lisp execution
- support unquote splicing in macro backquoting using the dot operator
- support "-quoted strings and store them as atoms
- also check flag `*a` in `evarg()` to prevent bleeding ghost bindings into dot-operator rest-of-arguments lists
- explain how to implement dynamic scoping for global functions, as in Common Lisp for example, to replace static scoping in Section "Constructing Lisp Expressions"
- introduce `isarg` to optimize variadic Lisp primitives supporting the dot operator, including empty lists (no arguments), as explained in section "Optimizing the Lisp Primitives"
- introduce `progn` that has historical roots in Lisp in section "Tail-Calls à Trois"
- make the `nth` and `nthcdr` argument order consistent with Common Lisp in Appendix E.2
- add `while` loop primitive to section "Tail-Calls à Trois"
- improve support for the dot-operator in backquotes e.g. ``(x . xs)` to handle more cases
- update functions `T`, `box`, `ord` and `equ` to use unions for safe type punning
- use a zero byte for `see` to signal the end of input or the end of an input line, instead of `\n`
- new section 17 "Mad Magical Macro Metaprogramming"
- new appendix E.6 "Macros" with some useful macro examples
- add an explanation with code to section 17.1 for early binding recursive function invocations